

Automotive software product lines for ECU software configuration: A systematic literature review

Yannick Lindebauer^{a,1,*}, Richard von Eisebeck^{b,1}, Thomas Vietor^a

^a Institute for Engineering Design, Technische Universität Braunschweig, Hermann-Blenk-Straße 42, Braunschweig 38108, Germany

^b Volkswagen AG, P.O. Box 1679/1583, Wolfsburg 38436, Germany

ARTICLE INFO

Edited by: Heiko Koziolk

Keywords:

Software product line engineering
Electronic control unit configuration
Automotive
Software-defined vehicle

ABSTRACT

In the automotive industry, vehicles are increasingly configured by software to accommodate individual customer preferences. This leads to a growing number of software variants and calibration parameters that must be managed consistently. Efficient handling of this variability benefits from the application of (systems and) software product line engineering (SPLE), as it offers structured mechanisms for reuse, traceability, and systematic variability management. These benefits are particularly relevant for electronic control unit (ECU) configuration, where maintaining consistency across features, parameters, and vehicle variants is crucial. However, existing approaches in industry, e.g. those relying on configurable bills of materials, indicate that SPLE has yet to see widespread adoption in the configuration of ECUs. Instead, proprietary approaches dominate, often lacking integration, transparency, and scalability. This study investigates the reasons why SPLE is not widely adopted for this use case. The underlying hypothesis suggests that industrial requirements are not accurately captured in academic research, making the implementation of scientific SPLE concepts difficult. To examine this assumption, a systematic literature review was conducted, analyzing relevant publications. The findings indicate a pressing need for closer collaboration between industry and academia to better identify challenges and requirements. Furthermore, current and emerging developments, such as software-defined vehicles (SDVs), require greater consideration in ECU configuration research. Our hypothesis was largely confirmed, indicating that SPLE research must be further extended and refined to meet practical ECU configuration needs. Accordingly, a concise, end-to-end methodology is needed to support SPLE-based calibration processes in SDV environments with increasingly decoupled hardware and software.

1. Introduction

The entire automotive industry is undergoing a fundamental transformation (Satou and Schmidt, 2024). This shift is no longer exclusively focused on transitioning to new drive systems but rather on the holistic evolution of the vehicle as a system (Streloke and Franke 2024). Customers increasingly demand deeper levels of vehicle customization, an expanding range of functionalities, and extensive digitalization (Zellmer et al., 2024b; Schindewolf et al., 2023). As a result, manufacturers are compelled to rethink their vehicles, making them software-driven and highly interconnected (Guissouma et al., 2019; Zellmer et al., 2024b). This transformation requires innovations in the electrical/electronic (E/E) architecture and the electronic control units (ECUs) used within modern vehicles (Satou and Schmidt, 2024; Streloke and Franke, 2024;

Schleicher, 2021).

This evolution is giving rise to software-defined vehicles (SDVs) which, in contrast to the current hardware-oriented approach, aim for seamless integration into the digital ecosystem, also in the context of the internet of things (IoT). SDVs encompass a wide range of functionalities, including vehicle-to-everything (V2X, also known as Car2X) communication, intelligent vehicle diagnostics based on real-time data, advanced driver assistance systems (ADAS) up to fully autonomous driving, as well as comfort features in infotainment, connectivity, and personalization (Satou and Schmidt, 2024; Singh and Gola, 2024; Laclau et al., 2025; Zellmer et al., 2024b).

A crucial aspect of SDVs is the capability to receive over-the-air (OTA) updates, enabling post-deployment software enhancements and security-critical updates without requiring to visit a service center

* Corresponding author.

E-mail address: yannick.lindebauer@tu-braunschweig.de (Y. Lindebauer).

¹ These authors contributed equally to this work.

(Guissouma et al., 2019; Schindewolf et al., 2023; Laclau et al., 2025; Schleicher, 2021). Additionally, the introduction of features on demand (FoD), also known as functions on demand, allows the activation or rental of specific functionalities, opening entirely new business opportunities (Cardozo et al., 2014; Schleicher, 2021).

As software functionalities - and consequently, the number of lines of code - continue to increase, the focus of value creation is shifting from hardware to software (Schindewolf et al., 2023). In vehicles, the software primarily consists of ECU software, i.e., embedded software. Modern vehicles already incorporate up to 150 million lines of code (Mallozzi et al., 2019; Staron, 2021). With the advancement of SDVs - for instance, in the domains of ADAS and, in particular, autonomous driving - not only is the volume of source code expected to increase further, but the amount of data generated by vehicles is also rising. This growth has direct implications for the software required to manage and communicate such data.

Historically, system requirements were primarily derived from hardware constraints, with software subsequently developed to meet these specifications. Under these new conditions, however, the software has become the defining element of vehicle functionality, which makes it necessary to adapt the hardware to the software requirements (Vincenzi et al., 2024).

Even before the emergence of SDVs, software variant management posed a significant challenge in the automotive domain. With the transition to SDVs, these challenges have become even more acute, especially given the complex requirements and constraints of the industry. One of the primary issues stems from the vast number of produced vehicles and, accordingly, the growing number of variants, which results in a drastically growing number of software variants. Software variability increases at a faster rate than hardware variability, as complexity expands both spatially and temporally due to additional variants and successive versions (Schindewolf et al., 2023).

In other industries - as seen at companies such as Siemens, Philips, Toshiba, HP, Bosch, and Boeing - (systems and) software product line engineering (SPLE) has already been established as a standard approach for comprehensive software variant management (Metzger and Pohl, 2014; Pohl et al., 2005; Clements and Northrop, 2002; van der Linden et al., 2007; ISO/IEC 26580:2021-04; ISO/IEC 26550:2015-12). Therefore, it also represents a promising solution for the automotive domain. However, it is crucial to acknowledge that the automotive domain presents unique and exceptionally complex challenges that distinguish it from other industries.

Contribution: In this paper, we conduct a systematic mapping study to investigate the extent to which SPLE is utilized in the automotive domain for the configuration of ECU software. We hypothesize that the adoption of SPLE in the automotive domain remains limited, with a comprehensive implementation largely absent. However, the configuration of ECU software for customer-specific vehicle variants could greatly benefit from an efficient management of software variants and related assets. A likely reason for this is that the requirements of the industry are not sufficiently addressed in academic research or are not given the necessary significance. We conclude that the SPLE research must be extended to meet industrial requirements and to enhance the applicability in real-world automotive contexts.

Outline: Section 2 provides background information on the challenges associated with software variant management in the automotive domain. Section 3 reviews related work. Section 4 presents the methodology of our systematic mapping study and details its execution. Section 5 presents the results of our study, answering the research questions and providing further insights. Section 6 discusses the threats to the validity of our research. Finally, Section 7 summarizes our findings.

2. Background

As described in the introduction, software variant management in the automotive domain is particularly challenging due to various difficulties and requirements. One significant challenge is the sheer number of variants and, consequently, the proliferation of software variants, which evolve along two dimensions - space and time (Kochanthara et al., 2022). A potential solution to this problem is the SPLE. The automotive domain represents an especially extreme application area, referred to by Wozniak and Clements (2015) as "mega-scale PLE".

In Section 2.1, we explore the extensive software variability in the automotive domain. This includes an explanation of how vehicle software can be configured and why this area has such unique challenges.

Building on this, Section 2.2 focuses on the widely recognized solution of SPLE. It illustrates how the configuration of ECU software in the automotive domain can be implemented within the SPLE.

2.1. ECU configuration & variant management in the automotive domain

Although the automotive domain does not inherently exhibit a higher degree of variability than sectors such as aerospace or industrial automation, it is distinguished by a unique combination of extensive system variability and high production volumes of complex, safety-critical products, requiring particularly efficient and scalable variant management approaches. This diversity is driven by factors such as the sheer number of vehicle models, individual customer preferences, different market regulations, and supplier strategies (Berger et al., 2020). While these drivers traditionally impacted hardware, the industry is increasingly shifting towards software-based customization (Agirre et al., 2020; Rumez et al., 2020). To meet customer demands for highly tailored vehicles, manufacturers offer extensive configuration options that go beyond physical features, such as wheels or interior equipment, to software-based features. These software adjustments can be made directly, through purchasable upgrades like CarPlay, or indirectly, such as when ordering heated seats, which require specific software configurations besides additional hardware. To manage this complexity, manufacturers rely on a universal software base, which can be adapted using parameters, datasets, and other mechanisms rather than developing custom software for every configuration (Heinisch et al., 2004; Lee and Han, 2009). Customer choices are consolidated into a build order, containing codes that define the software configuration (Berger et al., 2015). However, this process is particularly complex in embedded systems, where ECUs must be fully described, taking into account all relevant elements related to software, hardware, and configuration. Given the multitude of platforms and vehicle projects, software variability quickly scales into the millions, and software configurations must continuously evolve over time to accommodate new versions of hardware, software, and configuration components. Moreover, the challenge extends beyond a single ECU, requiring the coordination of an entire network of ECUs within a vehicle (Müller et al., 2006).

Another factor that makes variability management challenging in the automotive domain is the inherent complexity of modern vehicle systems. This complexity is further amplified by the ongoing shift toward centralized E/E architectures and the increasing use of high-performance computing platforms. As a result, many original equipment manufacturers (OEMs) are beginning to develop ECUs in-house. Consequently, they are now responsible for representing the full scope of variability themselves and for consolidating software variability within a single computing unit - rather than distributing it across multiple specialized subsystems, as was traditionally the case. Also, various systems are developed in parallel and must integrate seamlessly with

one another. The interdependencies between these systems are particularly difficult to manage in a complex product like a vehicle (Wozniak and Clements, 2015). One of the key challenges in this process is defining the configuration knowledge. This involves linking vehicle equipment to the corresponding configuration elements - a critical step that forms the foundation for successful ECU configuration.

The automotive industry is currently undergoing several profound transformations, including the transition to SDVs, the shift toward electromobility, and the advancement of autonomous driving. While the evolution toward SDVs primarily entails a shift in variability from the hardware to the software domain, the increasing integration of ADAS functionalities and other software-based services significantly amplifies the overall variability that must be managed. As a result, both the scope and complexity of software configuration are increasing, along with the demands placed on the supporting methods and tools. Additionally, unlike the traditional approach, where hardware dictated requirements and software was adapted accordingly, SDVs invert this relationship by making software the primary driver of functionality, with hardware adapting to software-defined constraints. This shift opens up new possibilities, such as microservices, architectures, and a potential decoupling of hardware and software, which could significantly impact how vehicle configurations are managed (Liu et al., 2022).

As the importance of software increases, regulatory compliance also becomes more critical. In the EU, UNECE Regulation R155 requires that all software components and ECUs must be documented, ensuring full traceability of their composition over time (Bohara et al., 2023). While this regulation introduces additional compliance efforts, it also presents new requirements and opportunities for the configuration management process. Alongside these regulatory requirements, configuration management in line with automotive SPICE (A-SPICE) (Levin et al., 2024) is essential. This process must not only manage software components but also include software configuration elements (like software versions, parameters, data sets, hardware IDs), ensuring all relevant elements and their changes are consistently tracked and documented (Ma et al., 2023).

2.2. Software product line engineering

SPLE addresses many of the challenges associated with managing high variability in software systems. It is understood as a framework that enables the reuse of software assets in highly variable environments. The overarching goal is to use resources more efficiently and to support synergies among various software assets (Metzger and Pohl, 2014) and thereby reduce development time and cost while improving product quality (Pohl et al., 2005).

SPLE aims to develop software applications using platform strategies and principles of mass customization. The use and development of platforms primarily involve planning for reuse and designing artifacts so that they can be deployed across multiple product variants (Pohl et al., 2005). These artifacts include, among others, requirements, architectural elements, and test artifacts as well as parameterization data.

We aim to utilize a variability management (VM) framework that is structured along two primary axes, each further divided into two sub-areas: domain engineering & application engineering and problem space & solution space. These two dimensions - domain and application engineering on one axis and problem and solution space on the other - together form a model that serves as the foundation for describing and managing variability (Apel et al., 2013). The framework operates as seen in Fig. 1 (Apel et al., 2013; Horcas et al., 2019): it captures technical variability and relates it to software assets. Based on specific input variants, the model generates the appropriate software assets, ensuring that every variant receives assets tailored precisely to its configuration (Böckle et al., 2005).

2.2.1. Domain engineering - problem space

In this phase, the product's variability is systematically described (Metzger and Pohl, 2014). In the automotive context, this means

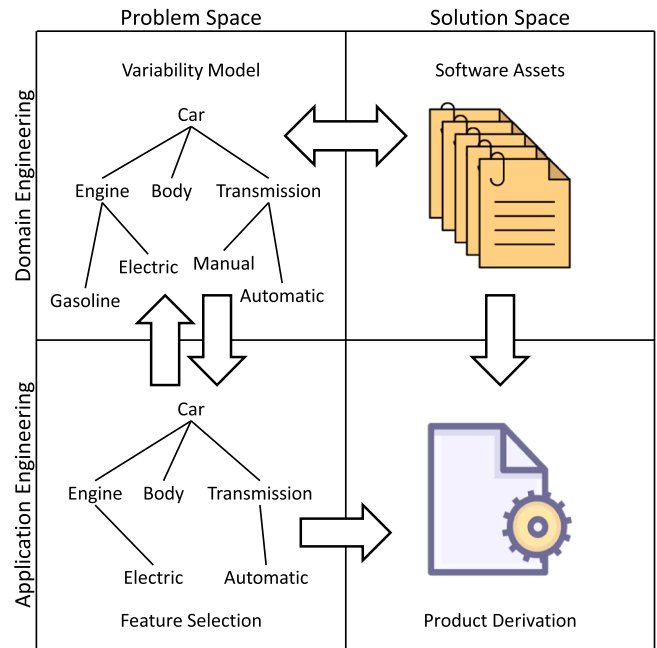


Fig. 1. VM Framework, adapted from (Apel et al. 2013; Horcas et al. 2019). The proposed framework captures technical variability and establishes a relationship with software artifacts. By processing specific input variants, the model generates the corresponding software artifacts, ensuring that each variant receives artifacts precisely tailored to its configuration.

modeling the variability of a vehicle based on its configurable features. For instance, when specifying the powertrain, a vehicle may offer different propulsion options such as gasoline, diesel, or electric. These options represent alternative feature values that can be selected during product configuration and must be captured accordingly in the variability model. To model this variability, various languages are available, including different types of feature models (Clarke et al., 2011; Thörn and Sandkuhl, 2009) or e.g. the orthogonal variability model (OVM) (Pohl and Metzger, 2006). Among these methodologies, feature models are the most widely used and are briefly explained below.

Feature models define features that represent specific properties, functionalities of a system (functional features) or environmental and operational conditions e.g. market region (context features). These features are organized hierarchically in a tree structure, where the hierarchy reflects relationships and dependencies between them. Different types of features determine how selections can be made within the model. Some features are optional, meaning they may or may not be selected. Others are alternative, where exactly one feature from a given group must be chosen. In some cases, multiple features from a group can be selected. This can be modeled more generally by defining the multiplicity of the feature group, for example using cardinalities such as 0..1 or 1..n to express optional, mandatory, or alternative selections. Another crucial aspect of feature models are the dependencies between features, which define how individual features interact with one another. Some dependencies follow an implication rule, meaning that if feature A is selected, feature B must also be included. Other dependencies enforce exclusion, ensuring that certain features, such as A and B, cannot be selected together (Thörn and Sandkuhl, 2009).

2.2.2. Application engineering - problem space

In this phase, a specific configuration variant is defined, serving as the basis for selecting corresponding software assets in the solution space, as seen in Fig. 1. This step determines the particular features and characteristics of the variant, ensuring that the corresponding software assets are derived accurately and consistently (Apel et al., 2013). For instance, within the automotive industry, this may refer to a

vehicle-specific build order, which is generated based on the individual customer configuration submitted via the online configurator. The resulting build order comprises a defined list of features that serve to unambiguously specify the intended vehicle variant.

2.2.3. Domain engineering - solution space

The domain engineering - solution space maps all collected software assets and their associated elements. This includes a wide range of components, from requirements and configuration elements to test cases. A key aspect of this phase is defining solution space configuration knowledge, which forms the foundation for linking the previously described variability from the problem space to the appropriate software assets (Boufedji et al., 2018; Metzger and Pohl, 2014). This knowledge explicitly specifies which software elements are associated with which variability characteristics. An illustrative example is the integration of calibration parameters into a hierarchically structured parameter model, which is systematically linked to the variability model described in Section 2.2.1. For instance, the parameter "maximum speed" can take on different values, such as 180 km/h. This specific value is then associated with a particular feature, such as an electric motor. This ensures that the maximum speed is automatically set to 180 km/h whenever an electric drivetrain is installed in the vehicle.

2.2.4. Application engineering - solution space

In the application engineering - solution space, the specific software elements for a given configuration are automatically derived and extracted. This process is based on the variant defined in the problem space, which is input into the variability model. Using the defined relationship knowledge, relevant software elements are identified and selected. This derivation ensures that all necessary assets are correctly tailored to the specific configuration (Apel et al., 2013). In the context of ECU configuration, the outcome would be a variant-specific configuration file derived from a concrete production order. This file contains the parameter values selected based on the input criteria described in Section 2.2.2. These parameters are explicitly linked to a specific vehicle variant and are used to perform the calibration and parameterization of the respective vehicle.

3. Related work

SPLE has become an established field in software engineering, and as such, a substantial body of research exists in this area. Several literature reviews on SPLE have also been conducted. However, we have not identified any studies specifically examining the application, requirements, and differences between industry and academia concerning the configuration of ECU software in the automotive domain with SPLE. While some literature reviews address SPLE within the automotive domain, they often focus on other specialized subfields. Zellmer et al. (2023) evaluate existing product structuring concepts, including SPLE. The study focuses on their applicability in the automotive domain but does not specifically address ECU software configuration. The authors conclude that the existing concepts lack practicality in an industrial context. Knieke et al. (2022) investigate comprehensive approaches to managing temporal evolution in automotive SPLE. The study finds that no holistic solutions currently exist to fully address this challenge. While temporal evolution is relevant to our research, we consider it as only one of several critical components. Our approach, therefore, extends beyond this narrower focus. In Hohl et al. (2017), a literature review on agile automotive SPLE for embedded software systems is presented. The goal was to identify connections to agile software development. The study reveals that agile SPLE approaches in the automotive domain have been rarely explored. However, this topic only marginally intersects with the research questions we aim to address. Another noteworthy research article (Espinell-Mena et al., 2024) partially overlaps with our area of interest by examining configuration management in SPLE. However, it does not specifically address the automotive domain and its unique

requirements. The literature also addresses the potential divergence between SPLE (models) and real-world industrial (automotive) practice. For example, Knüppel et al. (2017) analyze various feature models, including those from the automotive domain, and demonstrate that simple require and exclude constraints are insufficient to adequately capture the complexity of real-world variant models. They argue that research should incorporate additional real-world complexity, such as cross-tree constraints, to better reflect actual model structures and meet the demands of industrial applications. Other work, such as by Rabiser et al. (2018, 2019) and Becker et al. (2024), investigates the differences of academia and industry in SPLE research. While these studies do have examples of the automotive domain, they do not specifically address the use case of ECU configuration. Consequently, they do not identify challenges or reasons for the discrepancies in this particular application area.

While these previous studies touch on aspects related to our research questions, there are no comprehensive and up-to-date studies focusing specifically on the challenges and solutions for software configuration management in the automotive industry through SPLE. Our goal is to fill this gap by providing a detailed analysis that addresses the specific requirements and the intersections between industry and academia in this field.

4. Methodology

Software variant management in the automotive domain constitutes a considerable challenge that has increasingly attracted attention from both academic researchers and industrial practitioners. We hypothesize that the adoption of SPLE in the automotive domain for ECU configuration at the whole-vehicle level remains limited, with comprehensive implementations largely absent. However, it remains unclear what factors contribute to this situation and to what extent academic and industrial approaches and implementations of SPLE diverge or converge when applied to software variant management specifically in the automotive context. To address this gap, we conducted a systematic mapping study. This study follows the guidelines proposed by Kitchenham (2006), Kitchenham et al. (2011) and Kitchenham and Charters (2007) for conducting systematic research in the field of software engineering. An initial high-level overview of the research methodology is provided in Fig. 2.

4.1. Research questions

To gain clarity on the application of SPLE in the automotive domain for software variant management, we have formulated the following six research questions (RQs), which are categorized into three overarching areas:

RQ1 Academic Perspective:

RQ1.1: What are the academic use cases for SPLE in software variant management specific to the automotive domain?

RQ1.2: What challenges and research gaps remain in this academic context?

RQ2 Industrial Perspective:

RQ2.1: What are the industrial use cases for SPLE in software variant management within the automotive domain?

RQ2.2: What challenges and research gaps persist in the automotive industry?

RQ3 Comparison and Integration:

RQ3.1: How do academic and industrial use cases in the automotive domain differ in terms of application areas, tools utilized, and challenges faced?

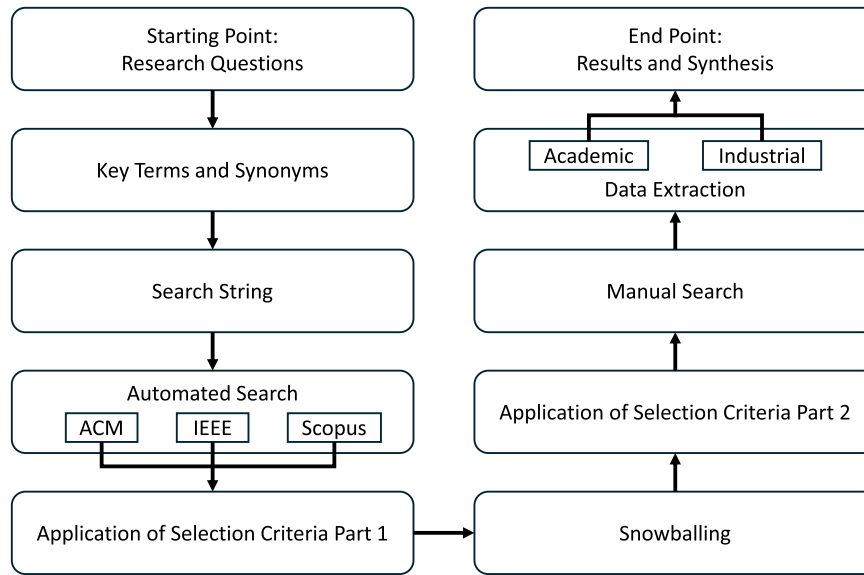


Fig. 2. Research Methodology.

The research methodology begins with the formulation of the research questions, followed by the identification of key terms and their synonyms. Based on these terms, a search string is constructed, which is then used to conduct an automated search in three academic databases: ACM, IEEE, and Scopus. Subsequently, the first phase of selection criteria application is performed. This is followed by a snowballing approach, after which the second phase of selection criteria application is conducted. To further complement the results, a manual search is carried out. The next step involves data extraction from both academic and industrial papers. Finally, the methodology concludes with the synthesis and presentation of the results.

RQ3.2: Are the automotive industry's requirements evident, well-defined, and sufficiently recognized by the academic community in the analyzed papers?

The research questions RQ1.1 and RQ2.1, as well as RQ1.2 and RQ2.2, address identical thematic issues, but approach them from different perspectives - from an academic viewpoint in one case and an industrial perspective in the other. RQ3.1 contrasts the use cases from both perspectives, analyzing their differences and similarities in terms of application areas, the tools employed, and the challenges encountered. Additionally, RQ3.2 establishes a further connection between academia and industry by investigating whether the requirements of the automotive industry are accurately and comprehensively reflected as boundary conditions within the academic community, both for current and future considerations.

4.2. Search strategy

Following the methodology recommended by Kitchenham and Charters (2007), we adhered to the established procedures for developing search terms and constructing a comprehensive search string. The selection and structuring of search terms represent a fundamental component of a systematic and reliable literature search (Kitchenham and Charters 2007; Zellmer et al. 2024a). To ensure conceptual consistency, logical relationships were defined between the terms, forming the basis of the final search string.

Inspired by Zellmer et al. (2024a), we formulated the three guiding questions, "What?", "How?", and "What for?", to systematically capture the scope, approach, and purpose of the investigated research. In contrast, frameworks such as PICO (Patient/Population, Intervention, Comparison, Outcome) would have structured the search differently. However, the three guiding questions were deliberately chosen, as they provide a more solid and comprehensible foundation for aligning the search strategy with the specific research objectives.

The guiding question "What?" aims to define the overarching scope of what is being examined and can thus be interpreted as a question regarding the domains under consideration. In contrast, the guiding question "How?" addresses the methods, tools, and frameworks to be

Table 1
Key terms and synonyms.

"What?"	"How?"	"What for?"
automotive; car; vehicle*; control unit; ecu; embedded	variabilit*; sple; ple; product line*	parameter handl*; parameter manag*; software calibrat*; software variant*; software config*

employed. The final guiding question, "What for?", clarifies the specific topic area and the particular problem we aim to address through the literature search.

Based on the research questions, these guiding questions are used to derive the key terms summarized in Table 1.

The transformation of search terms into a fully functional search string is achieved through the use of operators. In this process, the search terms associated with a specific guiding question are combined using the OR operator. In a subsequent step, the partial search strings generated for each guiding question are linked using the AND operator. Additionally, the use of wildcards enables the replacement of a single character (?) or zero to multiple characters (*). This allows for the identification of various variations of a search term while keeping the search string concise and organized. The resulting search string is as follows:

```

(("automotive" OR "car" OR "vehicle*" OR "control unit" OR "ecu" OR "embedded")
AND ("variabilit*" OR "sple" OR "ple" OR "product line*") AND ("parameter handl*" OR
"parameter manag*" OR "software calibrat*" OR "software variant*" OR "software
config*"))
  
```

This search string was applied to the full text search of papers available in the following three digital libraries.

ACM Digital Library: This library provides a comprehensive full-text collection of all ACM publications, including journals, conference proceedings, and technical magazines. Its focus is exclusively on the field of computing.

IEEE Xplore: A large database covering topics such as electronics, electrical engineering, and computer science. It includes journals,

conference proceedings, and technical magazines.

Scopus: Another extensive database offering a similar diversity of literature as the aforementioned libraries.

By deliberately selecting three well-established scientific databases, a substantial portion of the relevant literature in the research domain is covered. In addition, complementary searches were conducted using Google Scholar to ensure broader coverage. This procedure has already been successfully applied by Zellmer et al. (2023), Zellmer et al. (2024a) and Bernstein et al. (2025), among others. Similarly, other studies (Montes-Leon et al. 2026; Poy et al. 2025) adopt a targeted selection of data sources, thereby pursuing a different strategy compared to approaches that rely on a combination of two indexed and two non-indexed libraries.

4.3. Selection criteria

To extract papers that address the research questions, the following inclusion criteria (ICs) were applied:

- IC1: The paper deals with SPLE for automotive (including commercial vehicles) ECU software configurations.
- IC2: The paper has been published in a peer-reviewed journal, conference, or workshop.
- IC3: The paper is at least 4 pages long to ensure that it is not just a summary but a real contribution.
- IC4: The paper was published between 2004 and 2025.

An immediate exclusion of papers, however, is performed by applying the following exclusion criteria (ECs):

- EC1: The paper is only a Bachelor's or Master's thesis or a technical report.
- EC2: The paper contains incomplete or missing information on authors.
- EC3: The paper is published in a language other than English.
- EC4: The paper is a duplicate.

By applying the inclusion criteria and exclusion criteria - collectively referred to as selection criteria - we aim to identify the papers that provide the best possible qualitative basis for answering the research questions.

4.4. Data extraction

From each paper retrieved from the databases, we extracted the following citation and bibliographical information:

- Author(s)

- Document title
- Publication year
- Source and document type
- Author keywords
- Index keywords
- Number of pages

Additionally, during the thorough review of each paper's full text, further information was extracted. This includes a summary of the core content and contributions of the paper, as well as the domain to which the paper belongs. Furthermore, it was noted whether the document is a research paper or a case study. These results are presented, analyzed, and discussed in Section 5.

4.5. Conducting the review

The final automated search using the developed search string was executed on October 16, 2025, across the three databases: ACM Digital Library, IEEE Xplore, and Scopus. Preliminary exploratory searches had been conducted beforehand to iteratively refine and validate the search string. Our step-by-step approach to the literature search is illustrated in Fig. 3. Additionally, the figure shows the number of papers remaining as a result at the end of each step.

Using the search string and the selection criterion IC4, we initially started with a total of 2351 papers, which were progressively reviewed. In the first step, the title, abstract, and keywords of each paper were examined to identify those that addressed the research questions. Following this step, 63 papers remained. In the second step, a similar screening approach was applied to the introduction and conclusion sections, narrowing the selection to 37 papers.

In the subsequent step, forward and backward snowballing allowed us to add 5 papers to our review. After a full-text review and applying the remaining selection criteria, 26 papers remained. Additionally, we included 3 more papers as a result of a manual search. This brought the final number of papers selected for the study to 29.

To enhance the validity of our literature search, the intermediate results of each step were cross-checked by two of the authors before proceeding further. Any discrepancies regarding the relevance to the research questions were resolved through discussion. Only after reconciling all inconsistencies were the subsequent steps undertaken.

5. Results

This chapter answers the research questions. However, before doing so, general analyses and findings are presented to better understand the collected papers, their classification, and their content. Following this, the research questions are answered one by one, and the proposed hypothesis is evaluated. First, the recency of the analyzed papers is

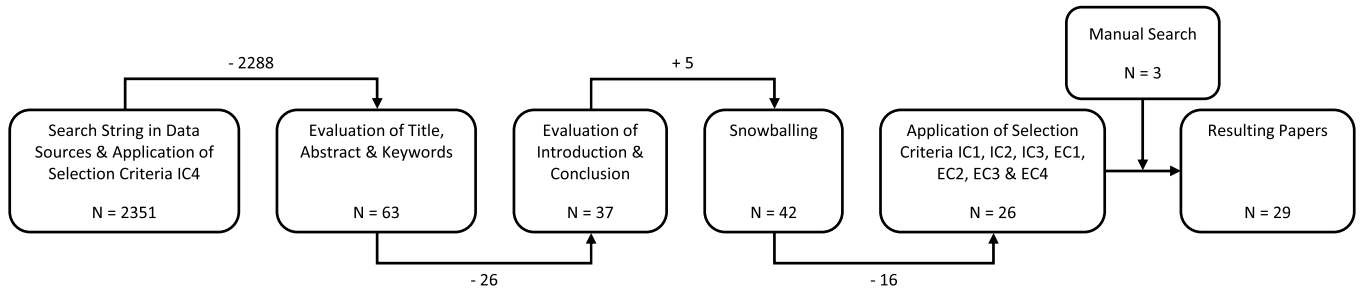


Fig. 3. Step-by-Step Approach of our Search Process.

Initially, a total of 2351 papers were identified based on the search string applied to the selected data sources, following the implementation of selection criterion IC4. After evaluating the titles, abstracts, and keywords, 63 papers remained. Subsequently, an assessment of the introductions and conclusions reduced this number to 37 papers. Through the application of the snowballing method, an additional five papers were identified, resulting in a total of 42 papers at this stage. The further application of selection criteria IC1, IC2, IC3, EC1, EC2, EC3, and EC4 led to a reduction to 26 papers. Finally, a manual search identified three additional papers, bringing the total number of selected papers to 29.

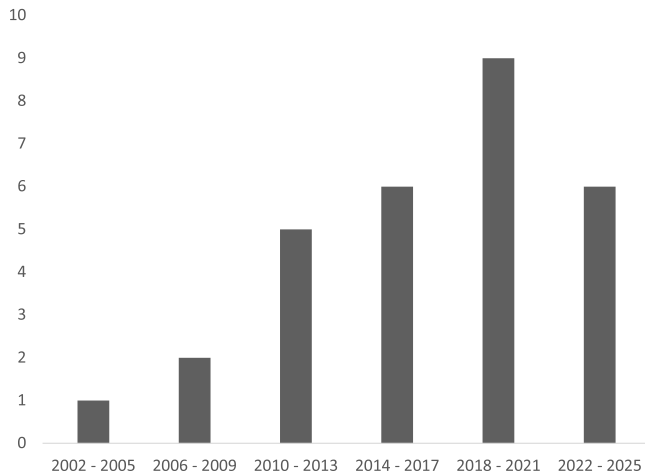


Fig. 4. Study Distribution by Year.

The figure presents the publication years of the studies. It highlights that the majority of the papers were published within the last decade, although some studies were also published between 2010 and 2013, and less between 2002 and 2009.

assessed to determine whether they primarily reflect past developments or current trends. For this purpose, the publication dates are displayed over the years in Fig. 4. The figure illustrates that the majority of the papers were published within the last decade. This highlights two points: first, the continued relevance of the topic, and second, that the literature review incorporates current scientific insights rather than focusing solely on older studies.

Table 2 reveals that they are almost evenly distributed between industry and academia. This results in a high proportion of case studies, as these often originate from industry, whereas academia might lack direct application opportunities. The findings highlight two key insights. First, the topic holds significant relevance for the industry, as considerable efforts are being made to integrate SPLE into ECU software configuration. Second, the application and validation of developed frameworks is significantly constrained without collaboration with an industry partner, as researchers often do not use large feature models for their empirical evaluation (Sundermann et al. 2024) or lack the necessary industrial context.

Table 3 assigns the identified publications to predefined research categories. These categories were designed to reveal thematic overlaps among the publications, capturing overarching thematic areas rather than individual terms. The analysis presented in Table 3 further indicates that research on SPLE often focuses on pure variant management, particularly on representing variability within domain engineering and the problem space. Moreover, SPLE research is frequently supported by model-based development approaches.

A broader evaluation, as shown in Fig. 5, highlights the emphasis placed by various papers within the VM framework. As Rabiser et al. (2018) also observed in general SPLE research, the emphasis remains largely on domain engineering, with comparatively little attention given to application engineering or the interaction between the two. This could be because the primary challenges in SPLE are often associated with domain engineering.

Another notable observation is that industrial case studies tend to address either holistic frameworks or primarily focus on domain engineering within the problem space. Academic papers, by contrast, often concentrate on specific subfields of SPLE, which reflects the inherent tendency of scientific research to address narrowly defined problems and explore targeted solutions.

Overall, it is evident that comprehensive frameworks and domain engineering within the problem space are either of the highest importance or represent the most significant challenges in SPLE.

5.1. Research question 1.1: academic use cases in the automotive domain

The application of SPLE in the automotive domain is widely discussed in academic literature, addressing various aspects for which suitable frameworks have been proposed. These frameworks can be categorized into the following four thematic groups:

- Variant management, model-based development, and SPL architecture (Leitner et al., 2011; Schindewolf et al., 2023; Bilic et al., 2019; Guissouma et al., 2021; Frank and Brenner, 2010; Otten et al., 2019; Mengi et al., 2009; Brink et al., 2014)
- OTA updates and safety & security (Bazzi et al., 2024; Guissouma et al., 2022)
- Software evolution and dynamic reconfiguration (Mauro et al., 2016; Röst et al., 2018)
- Clone-and-own and migration strategies (Ignaim et al., 2018)

Academic papers in this field typically adopt a narrow focus, addressing only one or two core areas of the VM framework as can also be observed in Fig. 5. In most cases, these studies concentrate on domain engineering, covering either the problem space, the solution space, or both. Only a few papers (Bilic et al., 2019; Guissouma et al., 2021; Frank and Brenner, 2010) introduce frameworks that also incorporate aspects of application engineering. Notably, only one study (Bilic et al., 2019) presents a comprehensive framework that covers all areas of the VM framework.

Furthermore, academic papers tend to provide a high-level overview of their proposed frameworks, often illustrating them with limited application examples.

The following subsections outline the core principles of academic frameworks, categorized by their respective thematic groups.

5.1.1. Variant management, model-based development, and SPL architecture

Leitner et al. (2011) propose an extension of the V-model by integrating a PLE environment tailored for automotive embedded systems. This approach enhances the traditional V-model with consistent variability handling, incorporating system architecture descriptions (EAST-ADL2), software unit and integration testing (Simulink-based), model-driven development (Matlab/Simulink), and software component deployment.

Schindewolf et al. (2023) introduce a refined variability model that combines SPLE with software configuration management (SCM). This enables buildability and upgradeability checks as well as dynamic allocation optimization. The refined variability model supports a common understanding of domain-specific concepts by combining variability modelling with dynamic assignments, thereby improving data reusability and consistency.

Bilic et al. (2019) outline an approach for handling SysML models within a PLE-based framework. It enables the enrichment of SysML models with variability information from variability management tools, allowing the generation of individualized system variants based on this information. The approach is implemented within a model-based PLE toolchain, where pure::variants represents the problem space, and the Modelio modeling tool covers the solution space. Variability information is exchanged using the variability exchange language (VEL).

Guissouma et al. (2021) propose a concept for managing variant-rich systems by deriving differences between variants in relation to a pre-defined basic variant using an extended delta model. The study combines contract-based design (CBD) with delta-based modeling, facilitating the use of variable contracts linked to specific features of the variability model or reusable components of the product line. The approach also connects to the open-source tool FeatureIDE.

Frank and Brenner (2010) address the challenge of identifying commonalities based on functionality, architecture, and naming conventions during variant identification, specification, and realization.

Table 2

List of relevant publications & further information.

ID	Paper	Origin	What is being presented?	What tools are being used?	Addressed Research Question
P01	(Leitner et al. 2011)	A	F	EAST-ADL, Simulink	RQ1.1, RQ3.1
P02	(Schindewolf et al. 2023)	A	F	-	RQ1.1, RQ1.2, RQ3.1, RQ3.2
P03	(Bazzi et al. 2024)	A	F	-	RQ1.1, RQ1.2, RQ3.1
P04	(Ignaim et al. 2018)	A	F	-	RQ1.1, RQ1.2, RQ3.1
P05	(Bilic et al. 2019)	A	F, T	SysML, pure::variants, VEL, Modelio	RQ1.1, RQ3.1
P06	(Gustavsson and Eklund 2010)	I	E	UML	RQ2.1, RQ3.1
P07	(Mauro et al. 2016)	A	F, T	HyVarRec	RQ1.1, RQ1.2, RQ3.1
P08	(Tischer et al. 2011)	I	E	-	RQ2.1, RQ3.1
P09	(Guissouma et al. 2021)	A	F	FeatureIDE	RQ1.1, RQ1.2, RQ3.1
P10	(Wozniak and Clements 2015)	I	E	DOORS, Gears	RQ2.1, RQ2.2, RQ3.1
P11	(Røst et al. 2018)	A	F, T	DSVL	RQ1.1, RQ3.1, RQ3.2
P12	(Dominka et al. 2017)	I	E, F	-	RQ2.1, RQ3.1
P13	(Steger et al. 2004)	I	E	-	RQ2.1, RQ3.1
P14	(Guissouma et al. 2022)	A	F	UPDATER	RQ1.1, RQ1.2, RQ3.1
P15	(Nishiura et al. 2018)	I	E	-	RQ2.1, RQ3.1
P16	(Frank and Brenner 2010)	A	F, T	ESCAPE	RQ1.1, RQ1.2, RQ3.1, RQ3.2
P17	(Ottén et al. 2019)	A	F	PREEvision	RQ1.1, RQ1.2, RQ3.1, RQ3.2
P18	(Mengi et al. 2009)	A	F, T	FeatureIDE	RQ1.1, RQ1.2, RQ3.1, RQ3.2
P19	(Brink et al. 2014)	A	F	-	RQ1.1, RQ1.2, RQ3.1 RQ3.2
P20	(Young et al. 2017)	I	E	Gears, Rhapsody, SysML	RQ2.1, RQ3.1
P21	(Berger et al. 2020)	I	E	-	RQ2.1, RQ2.2, RQ3.1
P22	(Bilic et al. 2020)	I	E	OVm, SysML, Integrity Modeler	RQ2.1, RQ2.2, RQ3.1
P23	(Manz et al. 2013)	I	E	-	RQ2.1, RQ2.2, RQ3.1
P24	(Pohl et al. 2018)	I	E, F, T	MIC, pure::variants, VEL	RQ2.1, RQ2.2, RQ3.1
P25	(Berger et al. 2015)	I	E	Gears	RQ2.1, RQ3.1
P26	(Esebeck 2025)	I	E	-	RQ2.1, RQ2.2, RQ3.2
P27	(Jost and Sinz 2024)	I	E	-	RQ2.1, RQ2.2
P28	(Giese et al. 2007)	I	F, T	UML; pure::variants	RQ2.1, RQ2.2
P29	(Kiltz and Becker 2025)	I	E, F	-	RQ2.1, RQ2.2, RQ3.2
Legend					

A = Academic (13), I = Industry (16); E = Experience Report (15), F = Framework (17), T = Tool / Toolchain (7).

The proposed industrial design tool ESCAPE supports managing a wide range of function network configurations and enables their graphical definition, manipulation, and analysis through systems modeling.

Ottén et al. (2019) propose a variant management approach for E/E systems within a model-based engineering process. It employs a hybrid characteristics model that integrates different variant management concepts, such as OVM and feature-oriented domain analysis (FODA). This enhances the consistent and formalized description of variability throughout the development process and facilitates early-phase product configuration. The approach is embedded in a model-based development environment for E/E systems, including the tool PREEvision.

Mengi et al. (2009) suggest a conceptual separation of problem space, solution space, and configuration knowledge. The problem space includes a cardinality-based feature model that manages variability, whereas the solution space consists solely of source code. The approach introduces a cardinality-based feature model linked to source code to shift work steps into the model by defining, managing, and implementing variation points. The implementation is realized as a plugin within the Eclipse framework.

Brink et al. (2014) propose managing hardware and software variability through two separate variability models. This decoupling allows easier exchange of hardware platforms and variants while ensuring compatibility between hardware and software during configuration testing. The hardware and software variability models are loosely coupled through properties represented as hierarchically ordered name-value pairs. A tool-based matching algorithm supports this approach.

5.1.2. OTA updates and safety & security

Bazzi et al. (2024) introduce a secure deployment approach for software modules in SDVs that supports OTA updates across different vehicle architectures. The approach employs a variability-rich scheme for software updates based on Merkle hash trees combined with digital signatures and feature modeling.

Guissouma et al. (2022) propose a system-oriented approach for

lifecycle management, design, implementation, verification, and deployment of OTA updates in vehicles. The framework includes a methodology for modular updates of variant-rich automotive systems, emphasizing continuous contract-based development, validation, and deployment of these updates. Additionally, it introduces a minimal-effort testing approach to ensure system safety after updates. The proprietary tool UPDATER (UPDateable Automotive Test dEmonstrator) is utilized to implement these methods and processes.

5.1.3. Software evolution and dynamic reconfiguration

Mauro et al. (2016) introduce a proprietary reconfiguration tool named HyVarRec, which evaluates the necessity of generating a new configuration based on the feature model, current configuration, and context information. If reconfiguration is required, the tool suggests a new configuration that is most similar to the original one. The study also introduces the concept of validity formulas (VFs), which are propositional formulas linking contextual information, features, and attributes to constrain feature selection.

Røst et al. (2018) address the software reuse challenge by integrating SPLE principles with available tools and industrial best practices. The approach supports the development and deployment of customized software adaptations and complex software products by leveraging hybrid variability. On the development side, a toolchain is introduced that utilizes SPL modeling techniques to manage feature combinations. The proprietary domain specific variability language (DSVL) is used within this toolchain. On the deployment side, a cloud-based toolchain complements the approach, operating in the IoT context to monitor a broad range of feature configurations in connected devices.

5.1.4. Clone-and-Own and migration strategies

Ignaim et al. (2018) propose a variability design model (VDM) that identifies and describes commonality and variability among customized cloned variants. Additionally, it introduces a mapping method that traces features back to their respective code locations by leveraging requirement specifications of the customized cloned variants. These

Table 3
Assignment of publications to research categories.

ID	Paper	Categories										
		Clown & Own	Dynamic Reconfiguration	Feature Analysis	Configuration Management	Model based Development	OTA Update	Safety & Security	Software Evolution	SPL - Architecture	SPL Merger & Migration	Variant Management
P01	(Leitner et al. 2011)					x				x		x
P02	(Schindewolf et al. 2023)		x			x						
P03	(Bazzi et al. 2024)						x	x				
P04	(Ignaim et al. 2018)	x									x	
P05	(Bilic et al. 2019)					x						x
P06	(Gustavsson and Eklund 2010)									x		
P07	(Mauro et al. 2016)		x									
P08	(Tischer et al. 2011)										x	
P09	(Guissouma et al. 2021)											x
P10	(Wozniak and Clements 2015)											x
P11	(Röst et al. 2018)								x			
P12	(Dominka et al. 2017)			x								
P13	(Steger et al. 2004)											x
P14	(Guissouma et al. 2022)						x	x				
P15	(Nishiura et al. 2018)	x									x	
P16	(Frank and Brenner 2010)					x						x
P17	(Otten et al. 2019)					x						x
P18	(Mengi et al. 2009)					x						
P19	(Brink et al. 2014)											x
P20	(Young et al. 2017)					x						x
P21	(Berger et al. 2020)											x
P22	(Bilic et al. 2020)					x						x
P23	(Manz et al. 2013)				x							x
P24	(Pohl et al. 2018)				x							x
P25	(Berger et al. 2015)											x
P26	(Esebeck 2025)				x				x			x
P27	(Jost and Sinz 2024)			x	x							
P28	(Giese et al. 2007)											x
P29	(Kiltz and Becker 2025)				x		x	x				x
		(2)	(2)	(2)	(5)	(8)	(3)	(3)	(2)	(2)	(3)	(17)

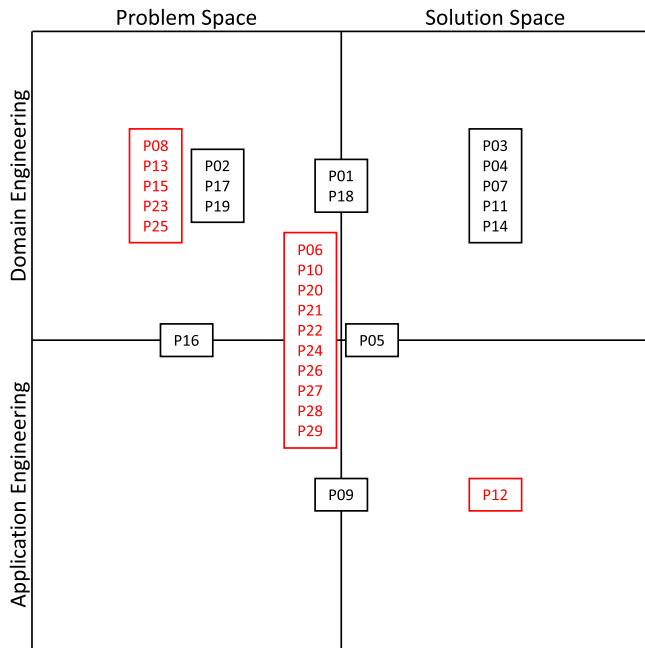


Fig. 5. Mapping of Research Papers onto the VM Framework according to the numbering (ID) shown in Tables 2 and 3. (Red Number = Industry; Black Number = Academic).

The figure illustrates the positioning of reviewed research papers within the SPLE framework. Each paper is placed according to the specific aspect of SPLE it addresses, providing a general overview of research coverage and potential gaps within the framework.

approaches collectively enhance the reuse potential of a set of customized cloned variants.

5.1.5. Key takeaways

Several academic publications examine the application of SPLE in the automotive domain. However, these studies often remain at a rather abstract level and typically focus on specific aspects of the VM framework. A holistic perspective that considers all core areas of the VM framework is frequently lacking.

Moreover, only a limited number of academic papers present a case study to evaluate their proposed framework. The case studies provided are generally implemented on a small scale and largely remain at a prototype level. As a result, existing research lacks practical validation through real-world industrial cases that would demonstrate the scalability and feasibility of these frameworks in an industrial environment. Consequently, the actual transferability of these frameworks to industry remains uncertain.

A common characteristic of several academic use cases for SPLE in software variant management specific to the automotive domain is the combined application of SPLE and model-based systems engineering (MBSE) approaches, as illustrated in Leitner et al. (2011), Bilic et al. (2019), Frank and Brenner (2010) and Otten et al. 2019). These relationships are evident in Table 2 through the tools and frameworks employed, as well as in Table 3 through the assigned categories. Within the academic community, this combined perspective has emerged as a widely accepted standard for the proposed frameworks, with feature models and SysML being the primary methodologies employed.

5.2. Research question 1.2: existing challenges and research gaps in academia

The remaining challenges and research gaps in this field are only briefly addressed in academic papers. At their core, these challenges appear to originate from the difficulty to transfer the proposed

frameworks into industrial practice and to implement these in real-world scenarios. While there are instances where academic frameworks have been applied in an industrial context, these cases are limited and typically focus on simplified and well-structured use cases.

Moreover, the reviewed academic papers rarely provide an in-depth discussion of the remaining challenges and research gaps following their respective studies. Instead, they primarily offer an outlook on the continuation of their presented research findings. In essence, they propose three main directions for future work. Firstly, they aim to investigate the applicability, required overhead, and scalability of the frameworks and methods in real industrial scenarios (Schindewolf et al., 2023; Bazzi et al., 2024; Ignaim et al., 2018; Guissouma et al., 2021; Guissouma et al., 2022; Frank and Brenner, 2010; Otten et al., 2019). Secondly, they emphasize the need for integration into the industrial workflow and early stages of the E/E development process (Schindewolf et al. 2023; Mauro et al., 2016; Guissouma et al., 2021; Otten et al., 2019; Mengi et al., 2009). Finally, they propose extending and refining the frameworks and methods based on these two aspects (Bazzi et al., 2024; Mauro et al., 2016; Guissouma et al., 2021; Guissouma et al., 2022; Brink et al., 2014).

A critical aspect that is entirely overlooked in discussions of challenges and research gaps is the shift towards SDVs and the profound transformations this entails for the automotive domain.

Additionally, it remains unclear whether the applicability of the research findings extends beyond the automotive domain. A transfer to other mobility domains such as aviation, aerospace, marine, and railway domains is not being sufficiently discussed or investigated. Similarly, potential applications outside the mobility sector are rarely considered. This indicates that research is conducted with a highly narrow focus. However, engaging in exchanges with other industries could enable significant benefits from cross-domain knowledge transfer and, for instance, provide the scientific foundation for standardization.

5.3. Research question 2.1: industrial use cases in the automotive domain

The application of SPLE in the automotive domain has been the focus of numerous industry case studies, each emphasizing different aspects. These studies can generally be divided into two main approaches. The first approach involves holistic SPLE concepts for ECU networks, often adopted by OEMs. These concepts describe how manufacturers configure and manage software across multiple ECUs within a vehicle network. Companies such as Scania, Volvo (Gustavsson and Eklund, 2010), Volkswagen (Esebeck, 2025), and Opel (Berger et al., 2015) have published shorter studies on their strategies, while General Motors (GM) (Young et al., 2017; Wozniak and Clements, 2015) and a truck manufacturer (Berger et al., 2020) have provided more detailed insights.

The second approach focuses on SPLE for specific products or individual ECUs, which is more commonly pursued by suppliers. In this case, the emphasis is placed on the product line of a single ECU or subsystem, often with a greater focus on methodological precision. Examples of companies applying this strategy include Bosch (Steger et al., 2004; Pohl et al., 2018; Tischer et al., 2011; Dominka et al., 2017), a supplier (Berger et al., 2020), Aisin Seiki (Nishiura et al., 2018), Daimler Truck (Manz et al., 2013), and Volvo Construction Equipment (Bilic et al., 2020).

This distinction is important because the challenges and requirements vary significantly between the two approaches. While OEMs must manage entire vehicle networks and ensure seamless software provisioning for production, suppliers concentrate on developing and maintaining variability within specific ECUs or subsystems. Despite these differences, both perspectives contribute valuable insights into the application and limitations of SPLE in the automotive industry.

The content of these publications varies widely. Most companies provide a broad overview of how SPLE is implemented within their organizations. Others focus more on specific technical aspects. Bosch (Steger et al., 2004; Pohl et al., 2018; Tischer et al., 2011; Dominka

et al., 2017), for example, examines the introduction, optimization, and analysis of SPLE, addressing aspects such as the integration of multiple SPLs and feature-based analysis. Aisin Seiki (Nishiura et al., 2018) presents a detailed account of the transition from clone & own to SPLE, highlighting improvements in feature model optimization.

These trends suggest that many companies are still in the early stages of SPLE adoption and tend to rely on holistic frameworks to gather feedback from the research community. At the same time, the lack of detailed disclosures in some publications implies that internal processes are deliberately not fully revealed. A recurring issue in these reports is their superficial nature, as they often outline general methodologies and frameworks without providing specific, realizable implementations of SPLE. This is particularly evident in Berger et al. (2015), Gustavsson and Eklund (2010), Young et al. (2017), Wozniak and Clements (2015) and Eisebeck (2025), which mention SPLE concepts in broad terms but fail to offer a detailed account of their practical application.

Bosch (Steger et al., 2004; Dominka et al., 2017; Pohl et al., 2018; Tischer et al., 2011) applies SPLE for the development and configuration of gasoline and diesel engine control units, with its product line recognized in the SPLC Hall of Fame. Their research extends to the integration of SPLs, feature-based analysis, and variant management.

ZF Friedrichshafen AG, in Kiltz and Becker (2025), presents an approach for leveraging PLE to address the increasing challenges posed by accelerated update cycles of ECU software and rising requirements such as cybersecurity compliance. To this end, they propose a Software Precalibration Pipeline, aimed at generating software variants including software parameters. The approach emphasizes modularization, distributed responsibilities, and scalable delivery of software variants.

GM (Young et al., 2017; Wozniak and Clements, 2015) presents SPLE as a holistic concept and examines the key requirements for its successful implementation in the automotive domain. While their study includes software configuration through calibration parameters, it lacks a detailed methodology for SPLE implementation. Opel (Berger et al., 2015), formerly part of GM, employs SPLE to manage vehicle variants using a bill-of-features, where software configuration is controlled via vehicle option codes, defining which software components are installed on ECUs. A similar approach is described by Jost and Sinz (2024) at Mercedes-Benz AG. There, the traditionally hardware-focused build orders are to be extended with additional software information and validated using SAT solvers. To represent the constraints formally, the authors propose a model based on Quantifier-Free Integer Difference Logic (QF-IDL).

Similarly, a truck manufacturer (Berger et al., 2020), manages software variability through feature-based configuration, linking features directly to source code for automated parameter selection. However, this approach relies on a flat feature database rather than advanced SPL methods such as feature models.

Eisebeck (2025) identifies key challenges for Volkswagen AG in the specific context of introducing SPLE for calibration parameter management and outlines how parameters are managed at Volkswagen using alphanumeric codes. However, the work remains at the level of problem identification and does not provide concrete solutions or methodological approaches to address the outlined challenges.

Another supplier (Berger et al. 2020) configures individual ECUs for customer-specific adaptations, utilizing feature databases and feature-to-code mappings with `#ifdef` statements.

Among OEMs, Scania (Gustavsson and Eklund, 2010) introduces a modular development approach and describes the use of PLE for architectural design based on UML, but without detailing software configuration.

The use of UML to model the behavior of highly variant systems was also explored by the former DaimlerChrysler Group (Giese et al., 2007), where software configuration was explicitly addressed as a use case. Using the example of a wiper control system, the authors demonstrate how, in the domain engineering phase, a feature model is created using Pure::Variants and subsequently linked to UML behavioral diagrams.

Based on a product configuration provided in an XMI file, variant-specific code is automatically generated and flashed.

Volvo (Gustavsson and Eklund, 2010) also applies PLE for architectural design, where software configuration is managed via a central binary configuration file deployed to the central ECU, from which individual ECUs retrieve their parameters. Some ECUs receive separate updates, and change management is emphasized as a key aspect.

In the commercial vehicle domain, Volvo (Bilic et al., 2020) has implemented SPLE for a small subsystem with automated software calibration, where human intervention is only required once to establish the link between features and calibration parameters. However, the scalability of this approach remains untested.

Aisin Seiki (Nishiura et al., 2018) successfully transitioned from clone & own to SPLE, significantly improving clarity and maintainability by reducing the number of features from 650 to 140.

Daimler Truck (Manz et al., 2013), on the other hand, introduced an SPL for engine control units, managing variability through an integrated feature model, where calibration parameters play a central role in defining variability.

Several industrial SPLE use cases exist in the automotive domain, but most publications remain high-level and describe implementations only superficially. This is particularly true for holistic SPLE approaches for ECU networks, where detailed methodological descriptions are largely missing. In many cases, it is simply stated that variability is managed via feature models and that calibration parameters are integrated through feature-to-code mappings, without explaining the exact implementation.

In contrast, use cases focused on individual ECUs or subsystems provide more detailed descriptions. These case studies present concrete methodologies that offer valuable insights for the application of SPLE in the automotive industry.

5.4. Research question 2.2: existing challenges and research gaps in industry

In nearly all industrial publications, the high number of variants is cited as a central challenge. On the one hand, this variability is the primary motivation for adopting SPLE, on the other hand, it remains a significant obstacle to its practical implementation (Manz et al., 2013). A frequently mentioned issue is the high initial effort required to fully model features in a feature model (Bilic et al., 2020; Pohl et al., 2018). This initial barrier is often difficult for companies to justify, as the return on investment only materializes at a later stage, while the upfront investment is substantial.

Beyond the sheer diversity of variants, vehicle complexity itself is a particularly challenging factor in the automotive industry. A major difficulty is inter-complexity - the dependencies between different systems and ECUs - which necessitates specialized approaches and significantly increases the effort required for consistent modeling (Eisebeck, 2025; Wozniak and Clements, 2015).

A core challenge arising from these factors is the linkage between features and software variability. This variability can manifest in different ways (e.g. in calibration parameters or as direct code variability). At the same time, dependencies between features must be accurately modeled, particularly when they span multiple hierarchy or abstraction levels (Pohl et al., 2018). They suggest researching the categorization of variability to structure the high number of variants and improve clarity.

Another frequently cited issue is implicit knowledge in SPLE development (Manz et al., 2013), particularly when building feature models. Many aspects of variability are not formally documented but instead reside in the minds of the engineers. The challenge is to establish systematic methods for capturing and integrating this expert knowledge into feature models.

Another challenge is the lack of clarity regarding the structural requirements of feature models in the context of SPLE and how such

models should be designed. There is a noticeable absence of methodological guidelines that OEMs could rely on when building their own variant models (Esebeck, 2025).

Even after successfully adopting SPLE, long-term maintenance and change management emerge as particularly difficult challenges (Manz et al., 2013). Tracking and documenting changes in the feature model is critical, especially when managing the relationship between spatial variability (variant management) and temporal evolution (version management). This highlights the need for a sustainable strategy for maintaining and evolving SPLE over time.

Furthermore, it is widely recognized that the successful introduction and use of SPLE requires strong management commitment (Pohl et al., 2018). Without a clear mandate and organizational anchoring, there is a significant risk that SPLE will not be consistently applied or maintained.

A further commonly mentioned barrier is the integration of SPLE into existing tool landscapes (Berger et al., 2020). A major factor is the lack of end-to-end toolchains for SPLE, which makes the industrialization of this methodology more difficult. Poor tool integration hinders seamless implementation into existing development processes and limits the efficiency of SPLE approaches.

In summary, several key challenges remain insufficiently addressed in industry practice, which are depicted in Table 4. These research gaps indicate that significant challenges remain in the practical implementation of SPLE in the automotive industry. A systematic investigation of these issues could help develop new solutions and improve both the adoption and long-term success of SPLE.

5.5. Research question 3.1: differences in application areas, utilized tools, and faced challenges

The examined academic and industrial use cases indicate many similarities. However, clear distinctions can be observed regarding their application areas, the tools used, and the challenges faced.

5.5.1. Application areas

The previously presented Fig. 5 provides an overview of which areas of the VM framework are addressed by the respective publications. A notable trend is the strong emphasis on domain engineering, whereas application engineering is rarely addressed. In cases where application engineering is considered, the respective studies typically provide a comprehensive examination of the VM framework, covering all four core areas.

A holistic analysis of the VM framework is predominantly conducted through industrial case studies. Nevertheless, individual industrial publications primarily focus on the problem space, whereas academic papers tend to cover either the problem space or the solution space - though the latter receives slightly more attention in academic research. This tendency arises from the fact that industrial case studies aim to develop comprehensive frameworks, while academic research often concentrates on specific subfields.

5.5.2. Faced challenges

As shown in Table 3 through the assigned research categories, both

academic and industrial publications primarily focus on the challenges of variant management, model-based development, and SPL architecture. In addition, industrial papers highlight configuration management as another key challenge within this domain.

Furthermore, both academic and industrial studies address challenges related to clone-and-own practices and migration strategies. However, the industrial perspective extends this focus by also considering the challenge of SPL merging.

One topic exclusively addressed in industrial publications is feature analysis. In contrast, academic research places significant emphasis on key topics such as OTA updates, safety & security, as well as challenges related to software evolution and dynamic reconfiguration.

5.5.3. Utilized tools

Distinctions can also be observed between the tools used in academic and industrial publications. These tools are listed in detail in Table 2. However, at first it is necessary to differentiate between languages and tools. While languages refer to modelling languages and exchange languages, tools include software solutions that can either be used independently or serve as platforms for modelling with specific languages.

Languages, with the exception of SysML, are primarily used to transfer variability information across different tools within a toolchain without loss of information. In contrast, SysML is a modelling language derived from UML, specifically designed for modelling software and complex systems. Both types of languages are utilized in academic as well as industrial publications.

In terms of tools, product line engineering (PLE) tools play a central role. These include Gears, pure::variants, FeatureIDE, and proprietary tools such as HyVarRec, UPDATER, and ESCAPE. While commercial tools and the open-source tool FeatureIDE provide a comprehensive range of functionalities - making them relevant in both academic and industrial studies - proprietary tools with a more limited scope are also found in academic research. These tools are typically tailored to specific tasks rather than offering broad functionality.

Beyond PLE tools, additional tools are employed, particularly within development environments for model-based systems engineering. Examples include PREEvision, Eclipse, Rhapsody, Integrity Modeler, and the Modelio modeling tool. These tools are integrated into both academic and industrial frameworks.

In addition to the aforementioned categories, further tools complement these frameworks. For instance, the requirements management tool DOORS is mentioned in an industrial study, while an academic publication refers to Simulink for simulation and analysis.

A notable trend in tool selection is that the industry tends not to disclose the use of proprietary tools, whereas such tools are frequently discussed in academic research. However, the absence of explicit references to proprietary tools in industrial publications should not be misinterpreted as an indication that these tools are not in use. Instead, it is likely that companies deliberately avoid disclosing details about in-house developments to prevent competitors from gaining insights into their proprietary methodologies.

The selection and concrete application of tools differ between academic research and industrial practice due to varying boundary

Table 4
Challenges and corresponding research contributions.

Challenge	Paper addressing the Challenge
High initial effort for feature modeling	-
Handling inter-complexity and managing dependencies	(Otten et al. 2019; Frank and Brenner 2010; Röst et al. 2018; Brink et al. 2014)
Mapping features to software variability	(Mengi et al. 2009)
Capturing and documenting implicit knowledge	(Frank and Brenner 2010)
Efficient change management	(Schindewolf et al. 2023; Röst et al. 2018; Frank and Brenner 2010; Otten et al. 2019)
Lack of tool integration	-
Traceability	(Otten et al. 2019)

conditions. Unlike in industry, academic research does not have access to the extensive, often proprietary datasets available within companies. Furthermore, academic research is not conducted within a corporate environment where numerous existing interfaces to established tools and data sources must be considered. Consequently, academic research neither necessitates nor realistically allows for the consideration of the complexity associated with industrial data foundations and interface infrastructures.

5.6. Research question 3.2: recognition of automotive requirements

This research question examines the extent to which academic studies contribute to solving industrial challenges and whether the developed concepts meet the specific requirements of the automotive domain.

It has already been observed that academia inherently struggles to test and validate its frameworks in real-world industrial environments. However, our hypothesis suggests that academic research only partially addresses actual industry needs and that practical problems may remain unrecognized. To test this hypothesis, two analyses were conducted: first, a comparison of the challenges identified in industrial case studies with the concepts proposed in academic publications, and second, an assessment of the extent to which recent developments - such as SDVs, regulatory requirements like UNECE R156 with its software update management system (SUMS), and other regulations related to OTA updates and configuration management - are addressed in academic research.

5.6.1. Comparison of industrial challenges vs. academic concepts

A visualization in [Table 4](#) illustrates which key industrial challenges have been addressed in scientific papers. However, this classification does not assess whether these challenges were effectively solved, but merely whether they were considered within the scope of the proposed solutions.

The results confirm the initial hypothesis to a large extent. Apart from general discussions on complexity and managing variability over time and space, many of the most pressing challenges faced by the industry are hardly addressed in academic research within the scope of ECU configuration.

Even when the temporal evolution of software is discussed, no methodological solutions or detailed models are presented. Many of the proposed concepts remain highly abstract, lacking practical implementation strategies and well-defined methodologies. The challenge of inter-complexity, referring to the dependencies between systems and ECUs, is mentioned in some academic publications, and certain concepts claim to address this issue. However, their depth of analysis remains insufficient. Notably, the form of inter-complexity described by [Wozniak and Clements \(2015\)](#) has never been fully explored or comprehensively addressed in the available literature.

Overall, it becomes evident that industry requirements are only partially considered in academic research. Furthermore, it is highly likely that specific industrial requirements remain unknown to academia. This underlines the necessity for stronger collaboration between industry and academia to align scientific concepts more closely with real-world industrial needs and ensure that proposed solutions are both relevant and applicable.

5.6.2. Comparison of industry developments vs. academic concepts

To further analyze whether academic research keeps pace with real-world developments, the addressing of three major trends in the automotive industry were examined: SDVs and the associated challenges, OTA updates and their requirements, and the integration of ECU calibration into configuration management with respect to SUMS and A-SPICE.

The analysis reveals that no scientific papers, whether from an industrial or academic background, thoroughly examine the impact of

SDVs on SPLE. One critical question remains unanswered: how does the transformation to a SDV affect SPLE and its use case for the configuration of ECU software? The shift towards SDVs introduces fundamental changes in system architecture, including the decoupling of hardware and software, as well as new possibilities for microservices, calibration, and diagnostics. Although some research discusses diagnostics in SDVs ([Boehlen et al., 2024](#); [Heithoff et al., 2023](#)), there is no clear link to the calibration of ECUs. If SDVs enable new approaches to calibration, this would have direct implications for SPLE usage, yet these potential transformations remain largely unexplored in current research.

In contrast to SDVs, OTA updates have been widely examined in this context. Several studies define the necessary requirements for OTA updates and present methodologies for their implementation. Additionally, research explores how OTA interacts with software in high-variability environments and which constraints must be considered. Unlike other areas, this suggests that OTA-related research is well aligned with industrial requirements and addresses relevant aspects of software life-cycle management in modern vehicles.

Configuration management, as defined by A-SPICE, covers the identification, control, and traceability of software components throughout the development process. In this context, the SUMS regulation plays a crucial role as it establishes specific requirements for managing software updates in vehicles. A-SPICE can help ensure compliance with SUMS by enforcing structured update management. Closely related to this is release management, which relies on baselines to track software versions and ensure consistency across updates. However, the role of baselines in release management within SPLE has received little attention in existing research. Despite the recognized importance of configuration management, a significant research gap remains in terms of traceability, documentation, and the integration of calibration parameters into SPLE. While SUMS and A-SPICE define general requirements, little focus has been placed on ensuring traceability between software updates and ECU calibration. This gap highlights the urgent need for further research, particularly regarding the documentation and validation of calibration processes within SPLE. Addressing these challenges would be essential for creating comprehensive and traceable configuration management solutions in the automotive domain. Studies such as ([Czarnecki et al., 2005](#)) address potential configuration processes in the context of ECU software configuration. They demonstrate that staged configuration constitutes a suitable methodology for managing domain complexity by dividing it into structured and manageable layers. In this approach, configuration is performed through a sequence of feature models, each representing a refinement stage, thereby enabling the stepwise elaboration of the final configuration. However, these works predominantly focus on the problem space within domain engineering and do not propose a comprehensive and implementable end-to-end methodology for real-world application.

This analysis confirms that academic research only partially addresses industrial challenges, which is likely due to its typically generic nature and lack of specific focus on industry needs ([Rabiser et al., 2018](#)). While high-level issues such as managing complexity and handling variant management are acknowledged, practical implementations are largely absent. Emerging industry trends, such as SDVs and their impact on SPLE, remain virtually unexplored. Although OTA updates have been more thoroughly researched, the integration of calibration processes into SPLE remains a major research gap. Emerging requirements such as OTA update capabilities or the Cyber Resilience Act are driving the need for systematic approaches like SPLE in parameter management ([Kiltz and Becker, 2025](#); [Esebeck, 2025](#)).

The findings emphasize the necessity of closer collaboration between academia and industry. Only by fostering stronger connections between both fields can scientific research be adapted to real-world requirements and contribute to practical solutions for SPLE implementation in the automotive domain.

6. Threats to validity

We acknowledge the threats that could impact the validity of our study. The threats to the validity of our study are discussed based on the four types of validity - construct validity, internal validity, external validity, and conclusion validity - as proposed by Wohlin et al. (2012).

6.1. Construct validity

Construct validity addresses the methodology presented in Section 4, specifically evaluating the quality of the approach we followed and applied to derive our results. The key question here is whether our study includes all relevant papers necessary to answer our research questions.

To ensure the quality of our search string, we verified that papers known beforehand to be important were present in the results of our database search. Additionally, at the conclusion of our study, we identified what we considered the most relevant sources from our final set of 29 papers and confirmed their presence in our database search results. Moreover, we sought feedback from several experienced researchers on the core literature we should consider. The presence of these suggested works in our results was also confirmed.

Despite these efforts, we had to supplement our final set of papers with 5 additional papers identified through snowballing and 3 papers from manual searches, resulting in the aforementioned total of 29 papers. However, we cannot entirely rule out the possibility that some relevant papers were overlooked by our methodology. In our view, four potential reasons could explain this:

While our search string was designed to identify variations of search terms and included synonyms, it cannot be guaranteed that all synonyms or alternative descriptions of relevant concepts in the papers were captured.

Our automated database search was limited to what we considered the three most relevant databases. Other databases and sources were excluded due to accessibility issues and other constraints. This limitation may have led to the omission of relevant papers.

Our systematic mapping study was conducted in parallel by two authors, with intermediate results compared and discussed. However, even with two researchers cross-checking each other's work, errors, such as the improper application of selection criteria, cannot be entirely ruled out given the sheer volume of papers reviewed.

Finally, it is also possible that some relevant papers were wrongly excluded during the initial stages of our step-by-step search process. In the early phases, we only reviewed the title, abstract, and keywords, followed by the introduction and conclusion, and considered the full texts only in subsequent steps.

6.2. Internal validity

Internal validity refers to the procedures applied during the extraction and synthesis of results and can be understood as the consistency and reliability of the findings. All results are inherently subject to the interpretation of the two authors conducting the study, which means that potential misinterpretations of the reviewed content cannot be entirely ruled out.

To mitigate the risk of subjective bias, we strictly adhered to the defined research methodology and performed regular cross-checks between two authors. For this purpose, a structured spreadsheet with a predefined schema was maintained for each author to systematically document and compare the extracted data, thereby supporting the consistency of the cross-check process. In addition, expert advice was obtained to further minimize the risk of subjective misinterpretations and to strengthen the overall validity of the findings.

6.3. External validity

External validity evaluates the applicability of the findings beyond the scope of our research. In this context, it is necessary to assess whether our results can also be applied to other use cases, i.e., to a broader scope. Specifically, the question arises: Is SPLE in software variant management within the automotive domain conceptualized differently compared to other mobility domains, and do distinct challenges emerge for those domains for the academic and industrial communities?

In principle, SPLE concepts can be extended to other mobility domains, such as aviation, aerospace, marine, and railway domains, from both academic and industrial perspectives. However, challenges and research gaps are less easily transferable, as each domain has its own specific requirements. These unique requirements influence the associated tool ecosystems and processes in a significant way.

One contributing factor may be the presence of domain-specific norms and standards. For commercial software-based aerospace systems, for instance, the DO-178C (Software Considerations in Airborne Systems and Equipment Certification) standard is applied. It defines, among other aspects, the framework for the software development process across various stages, including planning, requirements, design, implementation, verification, and configuration management. Furthermore, DO-178C also specifies the procedures for tool qualification (RTCA DO-178C).

6.4. Conclusion validity

Conclusion validity in the context of a systematic literature review refers to the extent to which the conclusions drawn from the review are accurate and reproducible. Publication bias may affect the conclusions, as statistically significant or positive findings are more likely to be published, potentially leading to an overestimation of the true effect size of scientific results. We are aware of this and take it into consideration to the extent possible.

7. Conclusion

This study conducted a systematic literature review across relevant databases to examine existing SPLE concepts for ECU software configuration in the automotive industry. The underlying hypothesis suggested that the requirements and challenges faced by the industry are only partially addressed in academic publications, making the full implementation of SPLE in the automotive domain for ECU configuration difficult to achieve. To verify this hypothesis, the identified publications were analyzed and categorized into industrial case studies and academic research. The comparison between industry-reported challenges and academically proposed solutions largely confirmed our hypothesis. It became evident that many industrial requirements are not fully recognized by academia, leading to concepts that are often only partially applicable to real-world challenges. Furthermore, recent developments, particularly the development of SDVs, have received little to no attention in scientific literature. Another significant finding is the absence of a comprehensive methodology detailing how ECU configuration can be systematically implemented using SPLE. While several publications mention this application and describe it in broad terms, there is a lack of detailed methodological frameworks that incorporate current regulatory and technical requirements.

Another important aspect is the absence of comprehensive concepts that provide a detailed description of how software configuration can be implemented while considering all aspects of configuration management within the framework of SPLE. Although this issue is mentioned,

no concrete methodology or implementation approach is provided. The lack of such detailed frameworks further complicates the practical application of SPLE in the automotive industry, as existing studies often remain at a conceptual level without addressing the specifics of real-world implementation.

We therefore conclude that the current VM framework alone does not sufficiently support the management of calibration parameters for extensive ECU networks in automotive applications. In particular, key aspects such as temporal configuration control and advanced configuration management need to be considered. While the integration of these capabilities might not have been part of the original scope of the VM framework, we believe that incorporating and expanding these functionalities is crucial to fulfill the practical needs and complex requirements of the automotive industry.

Future research should investigate whether solutions from other industries already exist that could be leveraged. The reviewed papers revealed few overlaps or references to other sectors, suggesting that cross-industry research holds significant potential for addressing current challenges.

Another particularly promising research avenue is the investigation of how ECU configuration can be managed using SPLE within a SDV environment, as the new opportunities and challenges arising from the decoupling of hardware and software remain largely unexplored.

In summary, this literature review underscores the key challenges associated with the application of SPLE in the automotive industry and identifies concrete areas for further research. The findings demonstrate that existing gaps can only be bridged through close cooperation between academic research and industrial practice, ensuring that future developments in SPLE align more closely with real-world needs.

Disclaimer

The results, opinions, and conclusions of this paper are not necessarily those of Volkswagen AG.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work the authors used Microsoft Copilot in order to improve readability and language of the manuscript. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

CRediT authorship contribution statement

Yannick Lindebauer: Writing – review & editing, Writing – original draft, Visualization, Validation, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Richard von Eisebeck:** Writing – review & editing, Writing – original draft, Visualization, Validation, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Thomas Vietor:** Writing – review & editing, Supervision, Project administration, Methodology, Funding acquisition.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Yannick Lindebauer reports financial support was provided by Volkswagen AG. Thomas Vietor reports financial support was provided by Volkswagen AG. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding sources

This work was supported by Volkswagen AG. We acknowledge

support by the Open Access Publication Funds of Technische Universität Braunschweig.

Data availability

No data was used for the research described in the article.

References

- Agirre, I., Onaindia, P., Poggi, T., Yarza, I., Cazorla, F.J., Kosmidis, L., et al., 2020. UP2DATE: safe and secure over-the-air software updates on high-performance mixed-criticality systems. In: Proceedings of the 2020 23rd Euromicro Conference on Digital System Design (DSD). 2020 23rd Euromicro Conference on Digital System Design (DSD). Kranj, Slovenia. IEEE, pp. 344–351, 26.08.2020 - 28.08.2020.
- Apel, S., Batory, D., Kästner, C., Saake, G., 2013. Feature-Oriented Software Product Lines. Concepts and Implementation. Springer, Berlin, Heidelberg.
- Böckle, G., Pohl, K., van der Linden, F., 2005. A framework for software product line engineering. In: Pohl, K., Böckle, G., van der Linden, F. (Eds.), Software Product Line Engineering. Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 19–38.
- Bazzi, A., Shaout, A., Di, M., 2024. A novel variability-rich scheme for software updates of automotive systems. IEEE Access. 12, 79530–79548. <https://doi.org/10.1109/ACCESS.2024.3409629>.
- Becker, M., Rabiser, R., Botterweck, G., 2024. Not quite there yet: remaining challenges in systems and software product line engineering as perceived by industry practitioners. et al. (Eds.). In: Cordy, M., Strüder, D., Pinto, M., Groher, I., Dhungana, D., Krüger, J. (Eds.), Proceedings of the 28th ACM International Systems and Software Product Line Conference. SPLC '24: 28th ACM International Systems and Software Product Line Conference. Dommeldange Luxembourg. New York, NY, USA. ACM, pp. 179–190, 02 09 2024 06 09 2024.
- Berger, T., Lettner, D., Rubin, J., Grünbacher, P., Silva, A., Becker, M., et al., 2015. What is a feature? In: Douglas, C.S. (Ed.), Proceedings of the 19th International Conference on Software Product Line. SPLC '15: 2015 International Conference on Software Product Lines. Nashville Tennessee. New York, NY, USA. ACM, pp. 16–25, 20 07 2015 24 07 2015.
- Berger, T., Steghöfer, J.-P., Ziadi, T., Robin, J., Martinez, J., 2020. The state of adoption and the challenges of systematic variability management in industry. Empir. Software Eng. 25 (3), 1755–1797. <https://doi.org/10.1007/s10664-019-09787-6>.
- Bernstein, S.; Rahman, A.; Sharifi, N.; Terbish, A.; MacNeil, S. (2025): Beyond the benefits: a systematic review of the harms and consequences of generative AI in computing education. doi: 10.1145/3769994.3770036.
- Bilic, D., Brosse, E., Sadovykh, A., Truscan, D., Bruneliere, H., Ryssel, U., 2019. An integrated model-based tool chain for managing variability in complex system design. In: Proceedings of the 2019 ACM/IEEE 22nd International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)2019 ACM/IEEE 22nd International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C). Munich, Germany. IEEE, pp. 288–293, 15.09.2019 - 20.09.2019.
- Bilic, D., Sundmark, D., Afzal, W., Wallin, P., Causevic, A., Amlinger, C., Barkah, D., 2020. Towards a model-driven product line engineering process. In: Jain, S., Gupta, A., Lo, D., Saha, D., Sharma, R. (Eds.), Proceedings of the 13th Innovations in Software Engineering Conference (formerly known as India Software Engineering Conference). ISEC 2020: 13th Innovations in Software Engineering Conference. New York, NY, USA. ACM, pp. 1–11. Jabalpur India, 27 02 2020 29 02 2020.
- Boehlen, B., Fischer, D., Wang, J., 2024. Diagnostics of automotive service-oriented architectures with SOVD. In: Boehlen, B., Fischer, D., Wang, J. (Eds.), Proceedings of the SAE Technical Paper Series. SAE 2024 Intelligent and Connected Vehicles Symposium. Shanghai, China. SAE International400 Commonwealth Drive, Warrendale, PA, United States (SAE Technical Paper Series).
- Bohara, R.; Ross, M.; Rahlfs, S.; Ghatta, S. (2023): Cyber Security and Software Update management system for connected vehicles in compliance with UNECE WP.29, R155 and R156.
- Boufedji, D., Guessoum, Z., Brandão, A., Ziadi, T., Mokhtari, A., 2018. Towards a MAS product line engineering approach. In: Ricci, A., Son, T C (Eds.), Amal El Fallah-Seghrouchni, Amal El Fallah-Seghrouchni, 10738. Springer International Publishing, Cham, pp. 161–179. Engineering Multi-Agent Systems(Lecture Notes in Computer Science).
- Brink, C., Kamsties, E., Peters, M., Sachweh, S., 2014. On hardware variability and the relation to software variability. In: Proceedings of the 2014 40th EUROMICRO Conference on Software Engineering and Advanced Applications (SEAA). Verona, Italy. IEEE, pp. 352–355, 27.08.2014 - 29.08.2014.
- Cardozo, N., Meuter, W., Mens, K., González, S., Orban, P.-Y., 2014. Features on demand. In: Collet, P., Wasowski, A., Weyer, T. (Eds.), Proceedings of the Eighth International Workshop on Variability Modelling of Software-Intensive Systems. VaMoS '14: The Eighth International Workshop on Variability Modelling of Software-Intensive Systems. Sophia Antipolis FranceNew York, NY, USA. ACM, pp. 1–8, 22 01 2014 24 01 2014.
- Clarke, D., Helvensteijn, M., Schaefer, I., 2011. Abstract delta modeling. Sigplan Not 46 (2), 13–22. <https://doi.org/10.1145/1942788.1868298>.
- Clements, P., Northrop, L., 2002. Software Product Lines. Practices and Patterns. Addison-Wesley (The SEI series in software engineering), Boston, San Francisco, New York, Toronto, Montreal, London, Munich, Capetown.

- Czarnecki, K., Helsen, S., Eisenecker, U., 2005. Staged configuration through specialization and multilevel configuration of feature models. *Softw. Process Imprv. Pract.* 10 (2), 143–169. <https://doi.org/10.1002/spip.225>.
- Dominka, S., Mandl, M., Ertl, D., Dubner, M., Schramm, F., 2017. Increasing test efficiency with automated feature-interaction-testing: feature testing of engine ECU software. In: Proceedings of the 2017 IEEE 28th Annual Software Technology Conference (STC). Gaithersburg, MD. IEEE, pp. 1–5, 25.09.2017 - 28.09.2017.
- Esebeck, R., 2025. Parameter management for automotive - ECU configuration via software product line engineering. et al. (Eds.). In: Luaces, M., Rodeiro, T., Greiner, S., Duarte, J., Yue, T., Yoshimura, K. (Eds.), Proceedings of the 29th ACM International Systems and Software Product Line Conference - Volume B. A Coruña Spain, 01 09 2025 05 09 2025. New York, NY, USA. ACM, pp. 1–5.
- Espinell-Mena, G.P., Carrillo-Medina, J.L., Galarza, E.E., Urbieto, M.M., 2024. A Systematic mapping of configuration management activities in software product line. *JUCS* 30 (11), 1484–1510. <https://doi.org/10.3897/jucs.110887>.
- Frank, A., Brenner, E., 2010. Model-based variability management for complex embedded networks. In: Proceedings of the 2010 5th International Multi-Conference on Computing in the Global Information Technology (ICCGI). Valencia, Spain. IEEE, pp. 305–309, 20.09.2010 - 25.09.2010.
- Giese, C., Schnieders, A., Weiland, J., 2007. A practical approach for process family engineering of embedded control software. In: Proceedings of the 14th Annual IEEE International Conference and Workshops on the Engineering of Computer-Based Systems (ECBS'07). Tucson, AZ, USA. IEEE, pp. 229–240, 26.03.2007 - 29.03.2007.
- Guissouma, H., Lauber, A., Mkaem, A., Sax, E., 2019. Virtual test environment for efficient verification of software updates for variant-rich automotive systems. In: Proceedings of the 2019 IEEE International Systems Conference (SysCon). Orlando, FL, USA. IEEE, pp. 1–8, 08.04.2019 - 11.04.2019.
- Guissouma, H., Kroger, J., Maelen, S.V., Sax, E., 2021. Extension of contracts for variability modeling and incremental update checks of cyber physical systems. In: Proceedings of the 2021 IEEE International Symposium on Systems Engineering (ISSE). Vienna, Austria. IEEE, pp. 1–8, 2021 IEEE International Symposium on Systems Engineering (ISSE)13.09.2021 - 13.10.2021.
- Guissouma, H., Hohl, C.P., Lesniak, F., Schindewolf, M., Becker, J., Sax, E., 2022. Lifecycle management of automotive safety-critical over the air updates: a systems approach. *IEEE Access* 10, 57696–57717. <https://doi.org/10.1109/ACCESS.2022.3176879>.
- Gustavsson, H., Eklund, U., 2010. Architecting automotive product lines: industrial practice. et al. (Eds.). In: Hutchison, D., Kanade, T., Kittler, J., Kleinberg, J.M., Mattern, F., Mitchell, J.C. (Eds.), Software Product Lines: Going Beyond, Software Product Lines: Going Beyond, 6287. Springer Berlin Heidelberg (Lecture Notes in Computer Science), Berlin, Heidelberg, pp. 92–105.
- Heinisch, C., Feil, V., 2004. Efficient configuration management of automotive software. In: Simons, M. (Ed.), Proceedings of the 2nd Embedded Real Time Software Congress (ERTS'04). Toulouse, France.
- Heithoff, M., Konersmann, M., Michael, J., Rumpe, B., Steinfurth, F., 2023. Challenges of integrating model-based digital twins for vehicle diagnosis. In: Proceedings of the 2023 ACM/IEEE International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C). Västerås, Sweden. IEEE, pp. 470–478, 01.10.2023 - 06.10.2023.
- Hohl, P., Ghofrani, J., Münch, J., Stupperich, M., Schneider, K., 2017. Searching for common ground: existing literature on automotive agile software product lines. In: Bendraou, R., Raffo, D., Li Guo, H., Maria Maggi, F. (Eds.), Proceedings of the 2017 International Conference on Software and System Process. ICSSP 2017: International Conference on the Software and Systems Process 2017. New York, NY, USA. ACM, pp. 70–79. Paris France, 05 07 2017 07 07 2017.
- Horcas, J.-M., Pinto, M., Fuentes, L., 2019. Software product line engineering: a practical experience. et al. (Eds.). In: Berger, T., Collet, P., Duchien, L., Fogdal, T., Heymans, P., Kehler, T. (Eds.), Proceedings of the 23rd International Systems and Software Product Line Conference - Volume A. SPLC 2019: 23rd International Systems and Software Product Line Conference. New York, NY, USA. ACM, pp. 164–176. Paris France, 09 09 2019 13 09 2019.
- Ignaim, K., Fernandes, J.M., Ferreira, A.L., Seidel, J., 2018. A systematic reuse-based approach for customized cloned variants. In: Proceedings of the 2018 11th International Conference on the Quality of Information and Communications Technology (QUATIC). Coimbra. IEEE, pp. 287–292, 04.09.2018 - 07.09.2018.
- ISO/IEC 26550:2015-12, 2015-12: software and systems engineering - Reference model for product line engineering and management.
- ISO/IEC 26580:2021-04, 2021-04: software and systems engineering - Methods and tools for the feature-based approach to software and systems product line engineering.
- Jost, F., Sinz, C., 2024. Handling automotive hardware/software co-configurations with integer difference logic. In: Kehler, T., Huchard, M., Teixeira, L., Birscher, C. (Eds.), Proceedings of the 18th International Workshop Conference on Variability Modelling of Software-Intensive Systems. VaMoS 2024. Bern Switzerland New York, NY, USA. ACM, pp. 103–111, 07 02 2024 09 02 2024.
- Kiltz, L., Becker, M., 2025. Navigating heterogeneous variability: strategies and insights from the industrial technology division of ZF Friedrichshafen AG. et al. (Eds.). In: Luaces, M.R., Rodeiro, T.V., Greiner, S., Duarte, J.G., Yue, T., Yoshimura, K. (Eds.), Proceedings of the 29th ACM International Systems and Software Product Line Conference - Volume A. SPLC-A '25: 29th ACM International Systems and Software Product Line Conference - Volume A. A Coruña Spain. New York, NY, USA. ACM, pp. 219–229, 01 09 2025 05 09 2025.
- Kitchenham, B.A., Charters, S. (2007): Guidelines for performing Systematic Literature Reviews in Software Engineering. Keele University and Durham University Joint Report.
- Kitchenham, B., 2006. Evidence-based software engineering and systematic literature reviews. et al. (Eds.). In: Hutchison, D., Kanade, T., Kittler, J., Kleinberg, J.M., Mattern, F., Mitchell, J.C. (Eds.), Product-Focused Software Process Improvement, Product-Focused Software Process Improvement, 4034. Springer Berlin Heidelberg (Lecture Notes in Computer Science), Berlin, Heidelberg, p. 3. p.
- Kitchenham, B.A., Budgen, D.P., Brereton, O., 2011. Using mapping studies as the basis for further research – a participant-observer case study. *Inf. Softw. Technol.* 53 (6), 638–651. <https://doi.org/10.1016/j.infsof.2010.12.011>.
- Knüppel, A., Thüm, T., Mennicke, S., Meinicke, J., Schaefer, I., 2017. Is there a mismatch between real-world feature models and product-line research? In: Bodden, E., Schäfer, W., van Deursen, A., Zisman, A. (Eds.), Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering. ESEC/FSE '17: Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on the Foundations of Software Engineering. Paderborn Germany. New York, NY, USA. ACM, pp. 291–302, 04 09 2017 08 09 2017.
- Knieke, C., Rausch, A., Schindler, M., Strasser, A., Vogel, M., 2022. Managed evolution of automotive software product line architectures: a systematic literature study. *Electronics (Basel)* 11 (12), 1860. <https://doi.org/10.3390/electronics11121860>.
- Kochanthara, S., Dajsuren, Y., Cleophas, L., van den Brand, M., 2022. Painting the landscape of automotive software in GitHub. In: Lo, D., McIntosh, S., Novielli, N. (Eds.), Proceedings of the 19th International Conference on Mining Software Repositories. MSR '22: 19th International Conference on Mining Software Repositories. Pittsburgh Pennsylvania. New York, NY, USA. ACM, pp. 215–226, 23 05 2022 24 05 2022.
- Laclau, P., Bonnet, S., Ducourthial, B., Li, X., Lin, T., 2025. Enhancing automotive user experience with dynamic service orchestration for software defined vehicles. *IEEE Trans. Intell. Transport. Syst* 26 (1), 824–834. <https://doi.org/10.1109/TITS.2024.3485487>.
- Lee, J.-C., Han, T.-M., 2009. ECU configuration framework based on AUTOSAR ECU configuration metamodel. In: Lee, G., Howard, D., Kang, J. J., Slezak, D., Nam Ahn, T., Yang, C.-H. (Eds.), Proceedings of the 2009 International Conference on Hybrid Information Technology. ICHIT '09: International Conference on Convergence and Hybrid Information Technology. Daejeon Korea. New York, NY, USA. ACM, pp. 260–263, 27 08 2009 29 08 2009.
- Leitner, A., Mader, R., Kreiner, C., Steger, C., Weiß, R., 2011. A development methodology for variant-rich automotive software architectures. *Elektrotech. Inftech* 128 (6), 222–227. <https://doi.org/10.1007/s00502-011-0001-0>.
- Levin, A., Stathatou, C., Lehmann, V., Benning, J.A., 2024. Handbuch Automotive SPICE 4.0. Grundlagen und Know-How Für die Praxis. 1. Auflage. Heidelberg: Dpunkt Verlag. Available online at https://content-select.com/index.php?id=bib_view&an=9783988901422.
- Liu, Z., Zhang, W., Zhao, F., 2022. Impact, challenges and prospect of software-defined vehicles. *Automot. Innov.* 5 (2), 180–194. <https://doi.org/10.1007/s42154-022-00179-z>.
- Müller, D., Herbst, J., Hammori, M., Reichert, M., et al., 2006. IT support for release management processes in the automotive industry. In: Hutchison, D., Kanade, T., Kittler, J., Kleinberg, J.M., Mattern, F., Mitchell, J. C., et al. (Eds.), Business Process Management, Business Process Management, 4102. Springer Berlin Heidelberg (Lecture Notes in Computer Science), Berlin, Heidelberg, pp. 368–377.
- Ma, Y., Gang, Y., He, D., Tan, X., 2023. Research on software project configuration management based on iterative development. In: Proceedings of the China SAE Congress 2021: Selected Papers. Singapore, 818. Springer Nature Singapore (Lecture Notes in Electrical Engineering), pp. 1237–1247.
- Mallozzi, P., Pelliccione, P., Knauss, A., Berger, C., Mohammadiha, N., 2019. Autonomous vehicles: state of the art, future trends, and challenges (Eds.). In: Dajsuren, Y., van den Brand, M. (Eds.), Automotive Systems and Software Engineering. Springer International Publishing, Cham, pp. 347–367.
- Manz, C., Stupperich, M., Reichert, M., 2013. Towards integrated variant management in global software engineering: an experience report. In: Proceedings of the 2013 IEEE 8th International Conference on Global Software Engineering. 2013 IEEE 8th International Conference on Global Software Engineering (ICGSE). Bari, Italy. IEEE, pp. 168–172, 26.08.2013 - 29.08.2013.
- Mauro, J., Niek, M., Seidl, C., Yu, I.C., 2016. Context aware reconfiguration in software product lines. In: Schaefer, I., Alves, V., Santana de Almeida, E. (Eds.), Proceedings of the Tenth International Workshop on Variability Modelling of Software-intensive Systems. VaMoS '16: Tenth International Workshop on Variability Modelling of Software-intensive Systems. Salvador Brazil. New York, NY, USA. ACM, pp. 41–48, 27 01 2016 29 01 2016.
- Mengi, C., Fuss, C., Zimmermann, R., Aktas, I., 2009. Model-driven support for source code variability in automotive software engineering. In: Proceedings of the 1st International Workshop on Model-driven Approaches in Software Product Line Engineering (MAPLE 2009), collocated with the 13th International Software Product Line Conference (SPLC 2009). San Francisco, California, USA, 557. Aug. 2009.
- Metzger, A., Pohl, K., 2014. Software product line engineering and variability management: achievements and challenges (Eds.). In: Herbsleb, J., Dwyer, M.B. (Eds.), Proceedings of the Future of Software Engineering Proceedings. ICSE '14: 36th International Conference on Software Engineering. Hyderabad India. New York, NY, USA. ACM, pp. 70–84, 31 05 2014 07 06 2014.
- Montes-Leon, S., Robles, G., Gonzalez-Barahona, J.M., 2026. Identification and classification of free, open source software licenses: a systematic literature review. *J. Syst. Softw.* 231, 112628. <https://doi.org/10.1016/j.jss.2025.112628>.
- Nishiura, Y., Asano, M., Nakanishi, T., 2018. Migration to software product line development of automotive body parts by architectural refinement with feature analysis. In: Proceedings of the 2018 25th Asia-Pacific Software Engineering Conference (APSEC). Nara, Japan. IEEE, pp. 522–531, 04.12.2018 - 07.12.2018.
- Otten, S., Glock, T., Hohl, C.P., Sax, E., 2019. Model-based variant management in automotive systems engineering. In: Proceedings of the 2019 International

- Symposium on Systems Engineering (ISSE). Edinburgh, United Kingdom. IEEE, pp. 1–7, 01.10.2019 - 03.10.2019.
- Pohl, K., Metzger, A., 2006. Variability management in software product line engineering. In: Osterweil, L.J., Rombach, D., Soffa, M.L. (Eds.), *Proceedings of the 28th International Conference on Software Engineering. ICSE06: International Conference on Software Engineering*. Shanghai China. New York, NY, USA. ACM, pp. 1049–1050, 20 05 2006 28 05 2006.
- Pohl, K., Böckle, G., van der Linden, F. (Eds.), 2005. *Software Product Line Engineering*. Springer Berlin Heidelberg, Berlin, Heidelberg.
- Pohl, T.B., Höchsmann, M., Wohlgemuth, P., Tischer, C., 2018. Variant management solution for large scale software product lines. In: Paulisch, F., Bosch, J. (Eds.), *Proceedings of the 40th International Conference on Software Engineering: Software Engineering in Practice. ICSE '18: 40th International Conference on Software Engineering*. Gothenburg Sweden. New York, NY, USA. ACM, pp. 85–94, 27 05 2018 03 06 2018.
- Poy, O., Moraga, M., García, F., Calero, C., 2025. Impact on energy consumption of design patterns, code smells and refactoring techniques: a systematic mapping study. *J. Syst. Softw.* 222, 112303. <https://doi.org/10.1016/j.jss.2024.112303>.
- Røst, T.B., Seidl, C., Yu, L.C., Damiani, F., Johnsen, E.B., Chesta, C., 2018. HyVar. Scalable hybrid variability for distributed evolving software systems. In: Ádám Mann, Z., Stolz, V. (Eds.), *Advances in Service-Oriented and Cloud Computing, Advances in Service-Oriented and Cloud Computing*, 824. Springer International Publishing/Communications in Computer and Information Science, Cham, pp. 159–163.
- Rabiser, R., Schmid, K., Becker, M., Botterweck, G., Galster, M., Groher, I., Weyns, D., et al., 2018. A study and comparison of industrial vs. academic software product line research published at SPLC. In: Berger, T., Borba, P., Botterweck, G., Männistö, T., Benavides, D., Nadi, S., et al. (Eds.), *Proceedings of the 22nd International Systems and Software Product Line Conference - Volume 1. SPLC '18: 22nd International Systems and Software Product Line Conference*. Gothenburg Sweden. New York, NY, USA. ACM, pp. 14–24, 10 09 2018 14 09 2018.
- Rabiser, R., Schmid, K., Becker, M., Botterweck, G., Galster, M., Groher, I., Weyns, D., 2019. Industrial and Academic Software Product Line Research at SPLC. et al. (Eds.). In: Berger, T., Collet, P., Duchien, L., Fogdal, T., Heymans, P., Kehler, T. (Eds.), *Proceedings of the 23rd International Systems and Software Product Line Conference - Volume A. SPLC 2019: 23rd International Systems and Software Product Line Conference*. New York, NY, USA. ACM, pp. 189–194, Paris France, 09 09 2019 13 09 2019.
- RTCA DO-178C, 2011-12: software Considerations in Airborne Systems and Equipment Certification.
- Rumez, M., Grimm, D., Kriesten, R., Sax, E., 2020. An overview of automotive service-oriented architectures and implications for security countermeasures. *IEEE Access* 8, 221852–221870. <https://doi.org/10.1109/ACCESS.2020.3043070>.
- Satou, I., Schmidt, M., 2024. Revamping the cockpit: latest trends in automotive e/e architecture, central computers, in-vehicle infotainment systems and displays. In: *Proceedings of the 2024 31st International Workshop on Active-Matrix Flatpanel Displays and Devices (AM-FPD)*. Kyoto, Japan. IEEE, pp. 1–4, 02.07.2024 - 05.07.2024.
- Schindewolf, M., Wittler, J.W., Kühn, T., Grimm, D., Sax, E., 2023. A model-based approach to automotive feature development for updates and upgrades. In: *Proceedings of the 2023 IEEE International Conference on Service-Oriented System Engineering (SOSE)*. Athens, Greece. IEEE, pp. 19–26, 17.07.2023 - 20.07.2023.
- Schleicher, M., 2021. Paving the way for the "Software Defined Vehicle". ELIV 2021. VDI Verlag, pp. 273–284.
- Singh, N., Gola, A., 2024. Empowering tomorrow's mobility: innovations in electric vehicle technology with IoT And AI integration. In: Bhutani, M., Gupta, P., Laxya, Sidharth (Eds.), *Advancing Innovation in Smart Systems, Energy, Materials, and Manufacturing: Unleashing the Potential of IoT, AI, and Edge Intelligence: Iterative International Publishers. Selfpage Developers Pvt Ltd*, pp. 141–159.
- Staron, M., 2021. *Automotive Software Architectures*. Springer International Publishing, Cham.
- Steger, M., Tischer, C., Boss, B., Müller, A., Pertler, O., Stolz, W., Ferber, S., et al., 2004. Introducing PLA at bosch gasoline systems: experiences and practices. In: Hutchison, D., Kanade, T., Kittler, J., Kleinberg, J.M., Mattern, F., Mitchell, J. C., et al. (Eds.), *Software Product Lines, Software Product Lines*, 3154. Springer Berlin Heidelberg (Lecture Notes in Computer Science), Berlin, Heidelberg, pp. 34–50.
- Strelake, L., Franke, J., 2024. Electrifying the road: disruptive shifts in automotive value creation. In: *Proceedings of the 2024 1st International Conference on Production Technologies and Systems for E-Mobility (EPTS)*. Bamberg, Germany. IEEE, pp. 1–8, 05.06.2024 - 06.06.2024.
- Sundermann, C., Brancaccio, V.F., Kuitert, E., Krieter, S., Heß, T., Thüm, T., 2024. Collecting feature models from the literature: a comprehensive dataset for benchmarking. et al. (Eds.). In: Cordy, M., Strüder, D., Pinto, M., Groher, I., Dhungana, D., Krüger, J. (Eds.), *Proceedings of the 28th ACM International Systems and Software Product Line Conference. SPLC '24: 28th ACM International Systems and Software Product Line Conference*. New York, NY, USA. ACM, pp. 54–65. Dommeldange Luxembourg, 02 09 2024 06 09 2024.
- Thörn, C., Sandkuhl, K., 2009. Feature modeling: managing variability in complex systems. In: Tolka, A., Lakhmi, C.J. (Eds.), *Complex Systems in Knowledge-based Environments: Theory, Models and Applications, Complex Systems in Knowledge-based Environments: Theory, Models and Applications*, 168. Springer Berlin Heidelberg (Studies in Computational Intelligence), Berlin, Heidelberg, pp. 129–162.
- Tischer, C., Müller, A., Mandl, T., Krause, R., 2011. Experiences from a large scale software product line merger in the automotive domain. In: *Proceedings of the 2011 15th International Software Product Line Conference (SPLC)*. Munich, Germany. IEEE, pp. 267–276, 22.08.2011 - 26.08.2011.
- van der Linden, F., Schmid, K., Rommes, E., 2007. *Software Product Lines in Action*. Springer Berlin Heidelberg, Berlin, Heidelberg.
- Vincenzi, M.; Pesé, M.D.; Bodei, C.; Matteucci, I.; Brooks, R.R.; Hasan, M. et al. (2024): Contextualizing security and privacy of software-defined vehicles: state of the art and industry perspectives.
- Wohlin, C., Runeson, P., Höst, M., Ohlsson, M.C., Regnell, B., Wesslén, A., 2012. Planning. In: Wohlin, C., Runeson, P., Höst, M., Ohlsson, M.C., Regnell, B., Wesslén, A. (Eds.), *Experimentation in Software Engineering*. Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 89–116.
- Wozniak, L., Clements, P., 2015. How automotive engineering is taking product line engineering to the extreme. In: Douglas, C.S. (Ed.), *Proceedings of the 19th International Conference on Software Product Line. SPLC '15: 2015 International Conference on Software Product Lines*. Nashville Tennessee. New York, NY, USA. ACM, pp. 327–336, 20 07 2015 24 07 2015.
- Young, B., Cheatwood, J., Peterson, T., Flores, R., Clements, P., 2017. Product line engineering meets model based engineering in the defense and automotive industries. et al. (Eds.). In: Cohen, M., Acher, M., Fuentes, L., Schall, D., Bosch, J., Capilla, R. (Eds.), *Proceedings of the 21st International Systems and Software Product Line Conference - Volume A. SPLC '17: 21st International Systems and Software Product Line Conference*. Sevilla Spain. New York, NY, USA. ACM, pp. 175–179, 25 09 2017 29 09 2017.
- Zellmer, P., Holsten, L., Leich, T., Krüger, J., 2023. Product-structuring concepts for automotive platforms: a systematic mapping study. et al. (Eds.). In: Arcaini, P., ter Beek, M.H., Perrouin, G., Reinhartz-Berger, I., Luaces, M.R., Schwanninger, C. (Eds.), *Proceedings of the 27th ACM International Systems and Software Product Line Conference - Volume A. SPLC '23: 27th ACM International Systems and Software Product Line Conference*. New York, NY, USA. ACM, pp. 170–181. Tokyo Japan, 28 08 2023 01 09 2023.
- Zellmer, P., Holsten, L., Krüger, J., Leich, T., 2024a. The terminology of automotive product-structuring concepts: a systematic mapping study. *IEEE Trans. Eng. Manag.* 71, 14974–14990. <https://doi.org/10.1109/TEM.2024.3463179>.
- Zellmer, P., Krüger, J., Leich, T., 2024b. Decision making for managing automotive platforms: an interview survey on the state-of-practice. In: d'Amorim, M. (Ed.), *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering. FSE '24: 32nd ACM International Conference on the Foundations of Software Engineering*. New York, NY, USA. ACM, pp. 318–328. Porto de Galinhas Brazil, 15 07 2024 19 07 2024.